

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Introducción a Android



Universidad
Rey Juan Carlos

¿Qué es Android?

Android es un sistema operativo móvil basado en el núcleo Linux y otro software de código abierto. Fue diseñado para dispositivos móviles con pantalla táctil, como teléfonos inteligentes o tabletas.

Posteriormente, se ha ampliado dando soporte a relojes inteligentes, automóviles o televisores a través de Wear OS, Android Auto y Android TV.



Historia

2003: Android fue creado por Rich Miner, Nick Sears, Chris White y Andy Rubin

2005: Google adquiere Android Inc. Se establece la Open Handset Alliance

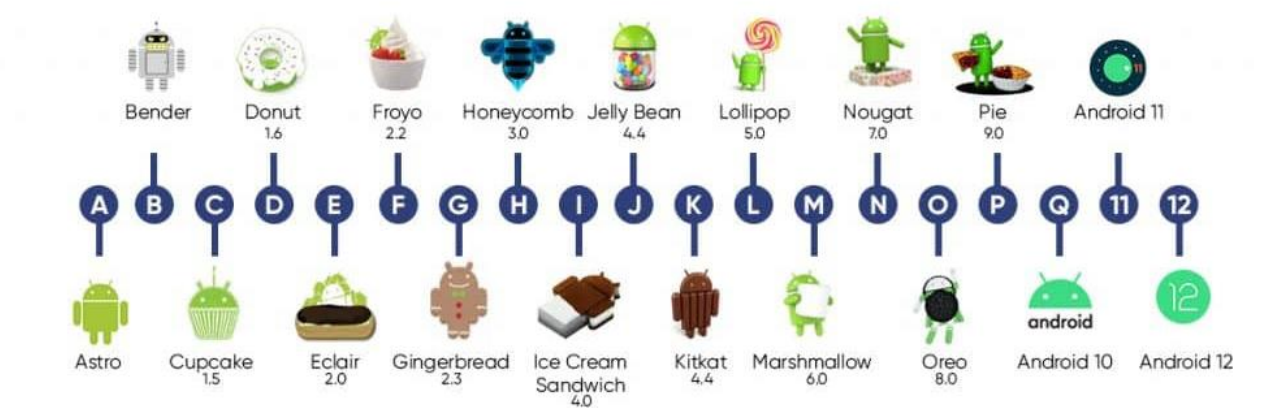
2008: Lanzamiento oficial de Android 1.0 Apple Pie

2009: Primer OS con un teclado en pantalla. Navegación con Google Maps

2017: Se introduce el soporte a Kotlin como lenguaje de desarrollo

2019: Kotlin es el nuevo lenguaje preferido para desarrollar apps Android

Presente: Android es el OS móvil más utilizado del mundo (>90% del mercado)



Características

- Código abierto
- Núcleo basado en el Kernel de Linux
- Diseñado para la manipulación directa de pantallas táctiles
- Existen diversas entradas alternativas que también son utilizadas para responder ante acciones de los usuarios:
 - Ej: giróscopos, acelerómetros, sensores de proximidad, voz, cámara, ...
- Amplísimo catálogo de aplicaciones gratuitas o de pago:
 - Cualquiera puede desarrollar apps y distribuirlas
 - Se pueden instalar directamente usando el fichero .APK
 - Google Play o stores alternativas

Entorno de desarrollo

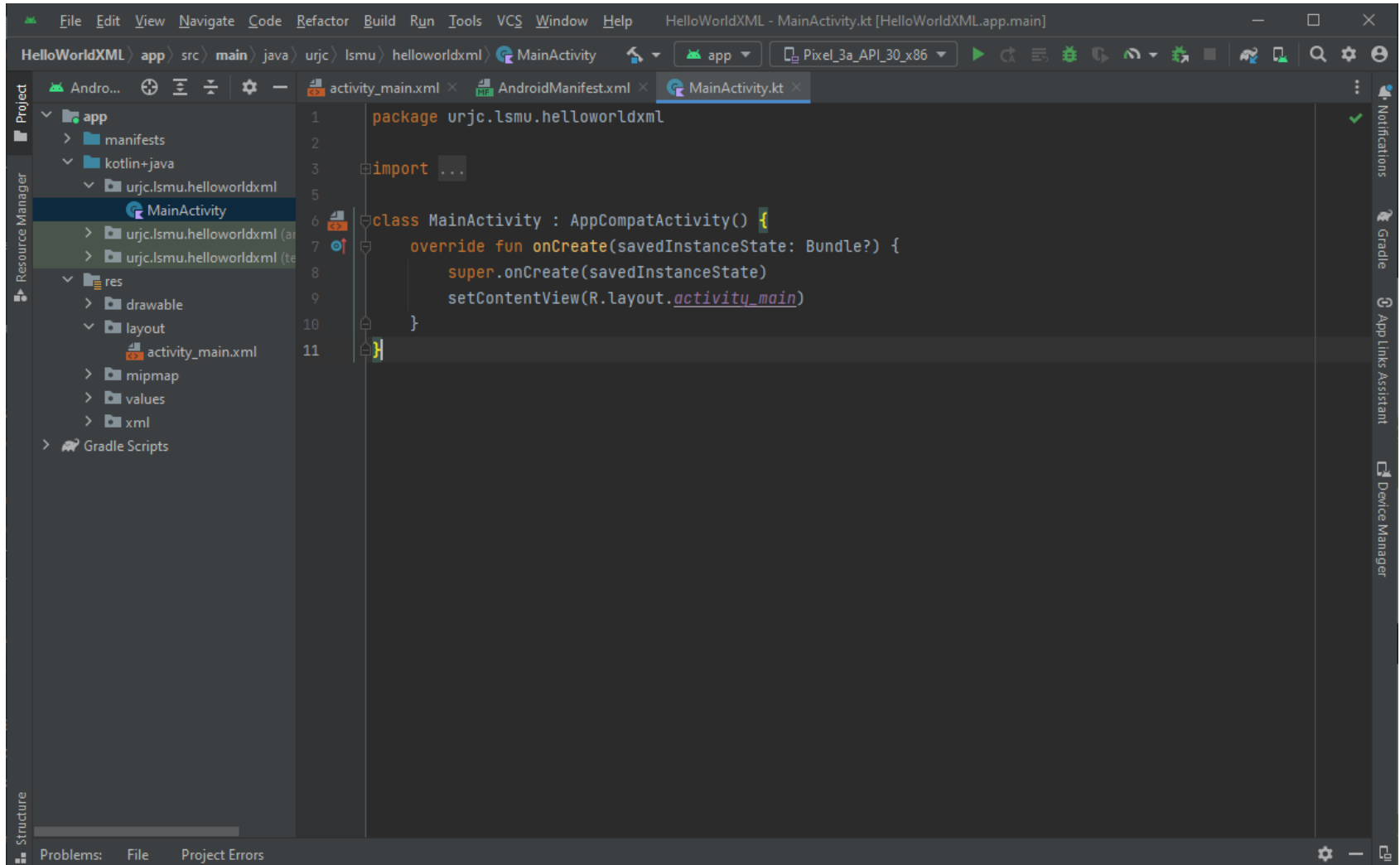
Android Studio es el entorno de desarrollo integrado (IDE) oficial del desarrollo de apps para Android.

- Programación en Java/Kotlin de la lógica de la aplicación
- Diseño de interfaces mediante XML/Compose
- Depuración de código
- Emulador para probar las aplicaciones en cualquier dispositivo
- Posibilidad de desarrollo en dispositivo propio vía cable o wifi
- Disponible para Windows, Mac y Linux



[Descargar Android Studio](#)

IDE: Android Studio





**¡A por tu primera
aplicación!**

(en la guía de Aula Virtual)

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

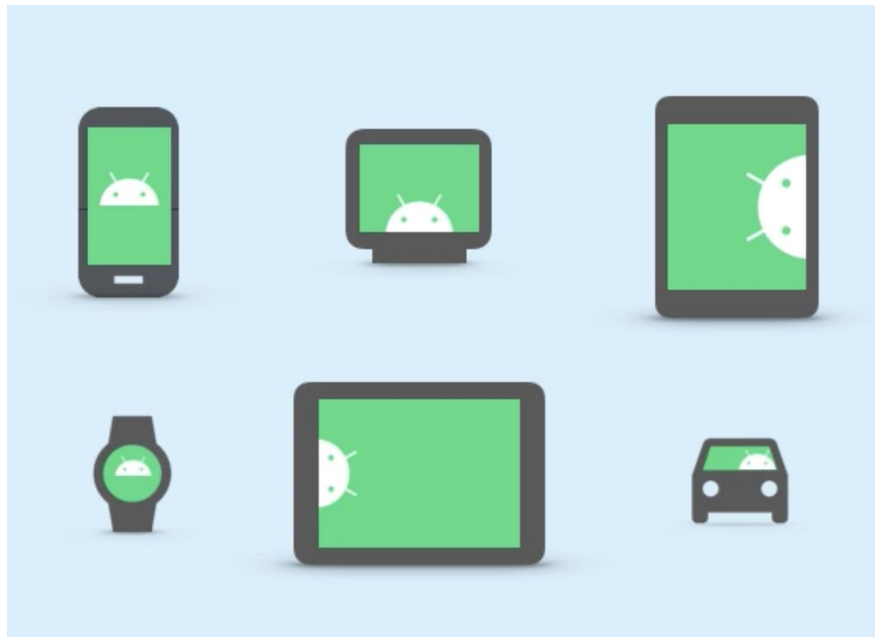
Layouts: diseñando interfaces



Universidad
Rey Juan Carlos

Diseño de interfaces en Android

- Los dispositivos Android vienen en muchos factores de forma diferentes.
- Cada vez hay más píxeles por pulgada en las pantallas de los dispositivos.
- Los desarrolladores necesitan poder especificar dimensiones de diseño que sean coherentes en todos los dispositivos.

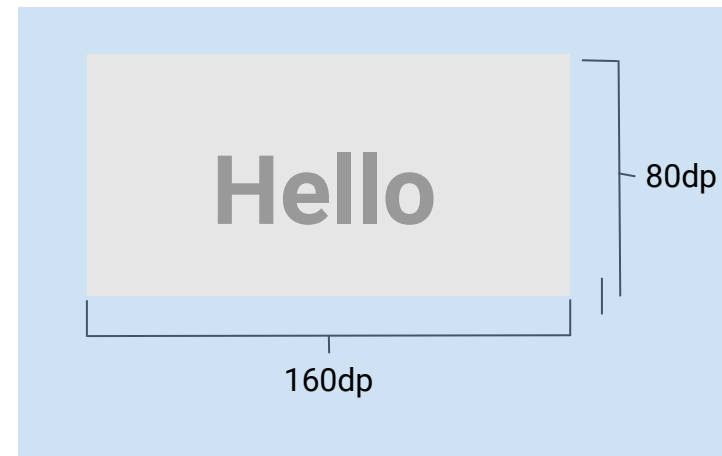


Píxeles independientes de la densidad (dp)

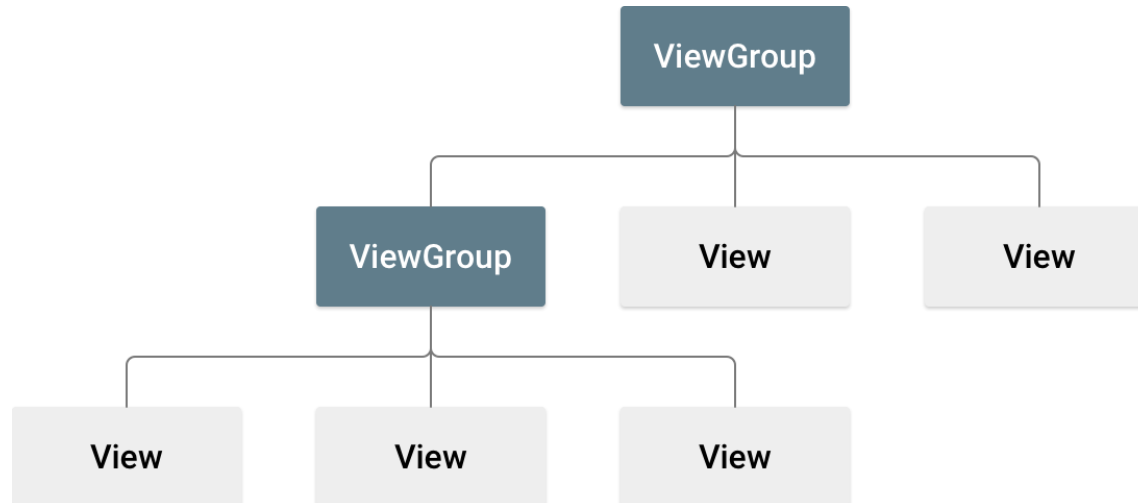
Utilice dp cuando especifique tamaños en su maqueta, como la anchura o la altura de las vistas.

- Los píxeles independientes de la densidad (dp) tienen en cuenta la densidad de la pantalla.
- Las vistas de Android se miden en píxeles independientes de la densidad.

Density qualifier	DPI estimate
ldpi (mostly unused)	~120dpi
mdpi (baseline density)	~160dpi
hdpi	~240dpi
xhdpi	~320dpi
xxhdpi	~480dpi
xxxhdpi	~640dpi



Organización de un layout



View: se suelen llamar widgets, y muestran los objetos que el usuario ve en pantalla. Ejemplos: Button, Textview.

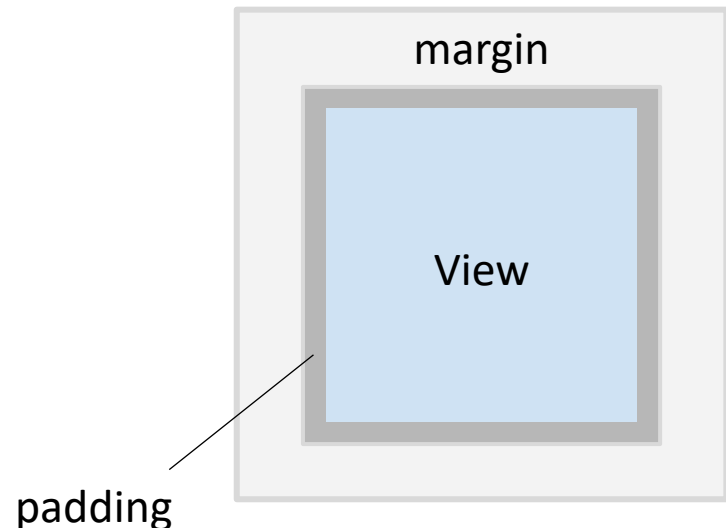
ViewGroup: conocidos como diseños, definen estructuras o contenedores en los que organizar elementos *View* y otros *ViewGroup*. Ejemplos: ConstraintLayout

Se pueden declarar en los archivos de layout XML, o crear en tiempo de ejecución.

Parámetros de diseño más comunes

- **id**: identificador único del elemento en la interfaz. Las referencias a la ID de un elemento se usan para asociarlo a los objetos declarados en las Activities (usando *findViewById*), así como al definir las relaciones con otros objetos de la interfaz.
- **layout_width/layout_height**: definen el tamaño del elemento con medidas exactas (no recomendable) o las constantes *wrap_content* y *match_parent*.
- **layout_something**: otros parámetros de diseño que definen márgenes, padding, o relaciones de posición entre elementos.

```
<Button  
    android:id="@+id/my_button"  
    android:layout_width="wrap_content"  
    android:layout_height="match_parent"  
    android:text="@string/my_button_text" />
```

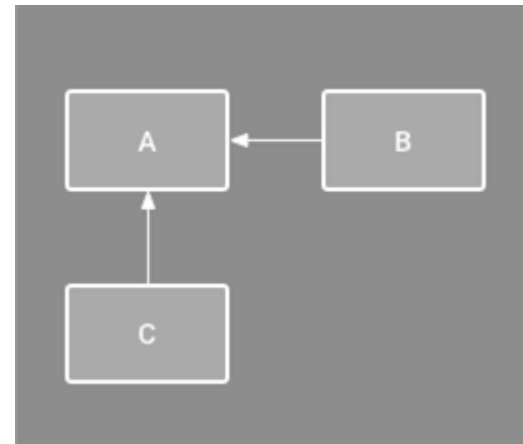


ConstraintLayout

ConstraintLayout

Es un *ViewGroup* que organiza los elementos hijos en posiciones relativas. Dicha posición puede definirse mediante restricciones con elementos del mismo nivel, o respecto a la vista padre.

El uso de *ConstraintLayout* permite diseñar interfaces de manera simplificada manteniendo una jerarquía más plana, evitando anidar *ViewGroups* y, por tanto, mejorando el rendimiento.



Posicionamiento en un ConstraintLayout

layout_constraint<Origen>_to<Destino>: parent | viewID

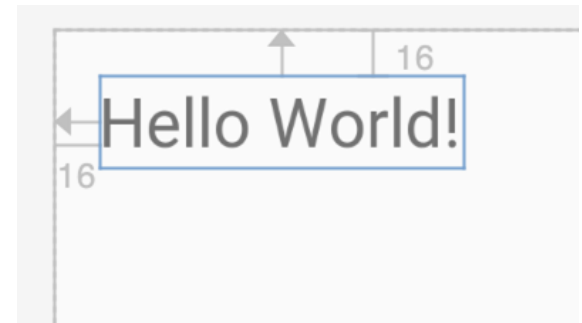
Establece el alineamiento del lado <Origen> del elemento que tiene el atributo respecto al lado Destino del elemento relativo sobre el que se posiciona.

- Origen | Destino = *Start* (izq), *End* (dcha), *Top* (arriba) y *Bottom* (abajo)

Ejemplo de atributos en un TextView:

```
app:layout_constraintTop_toTopOf="parent"
```

```
app:layout_constraintLeft_toLeftOf="parent"
```

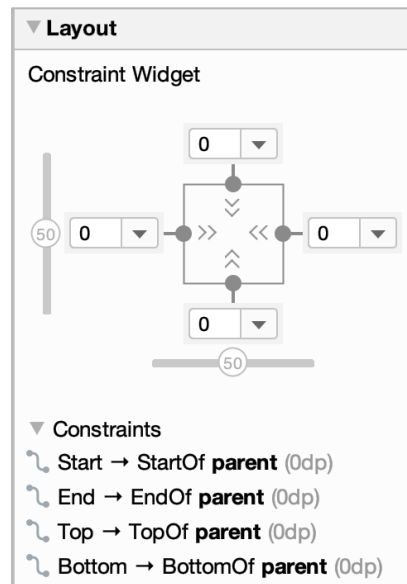


parent hace referencia al contenedor (ViewGroup) que engloba a un elemento. En este caso, al *ConstraintLayout*.

Tipos de restricciones

Usando el Layout Editor junto al Constraint Widget del panel de atributos (a la derecha) se puede ajustar el posicionamiento de un elemento de forma intuitiva.

La vista respecto a la que se posiciona el elemento seleccionado se puede ajustar con el ratón desde el Layout Editor. Hay tres tipos de restricción:



Fixed



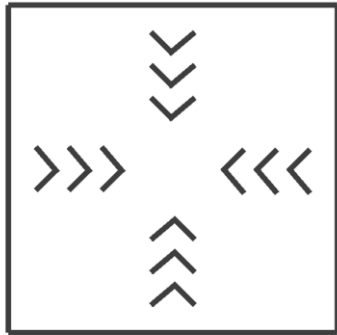
Wrap content



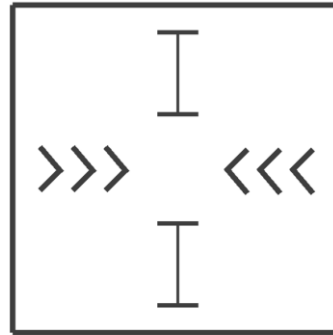
Match constraints

Tipos de restricciones

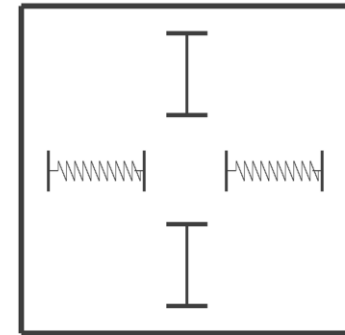
El ancho y alto de los elementos se puede ajustar para que se ajuste al contenido, tenga un valor fijo, o se adapte para cumplir las restricciones de la vista.



layout_width `wrap_content`
layout_height `wrap_content`

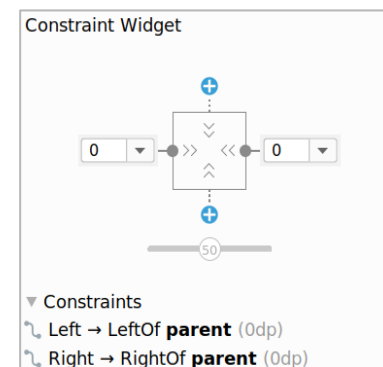


layout_width `wrap_content`
layout_height `48dp`



layout_width `0dp(match_constraint)`
layout_height `48dp`

Si las restricciones opuestas están fijadas y no se les asigna una distancia (0dp), el elemento se centra.



Ejemplo de ConstraintLayout

activity_main.xml

```
<androidx.constraintlayout.widget.ConstraintLayout
```

```
...
```

```
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context=".MainActivity">
```

```
...
```

```
<TextView
```

```
...
```

```
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:layout_constraintBottom_toBottomOf="parent"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toBottomOf="@+id/button"/>
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

Ajuste relativo a la vista padre
a la izquierda, arriba y abajo

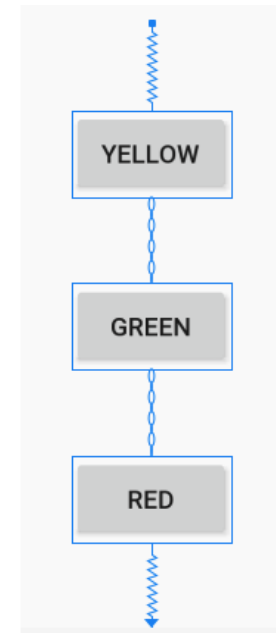
Ajuste relativo a
otro elemento
dentro del
ConstraintLayout

Cadenas de elementos

- Permiten posicionar unos elementos respecto a otros
- Pueden enlazarse vertical u horizontalmente
- Proporcionan la funcionalidad anteriormente provista por *LinearLayout*

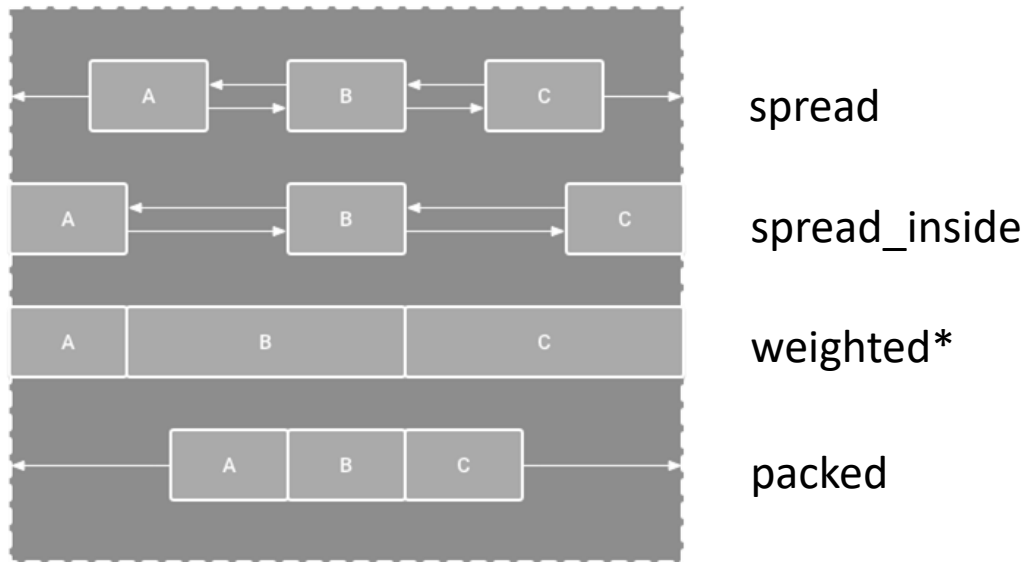
Creación de una cadena desde el Layout Editor

1. Seleccionar los elementos que formen la cadena
2. Hacer clic con el botón derecho y seleccionar *Chains*
3. Crear una cadena horizontal o vertical



Tipos de cadenas de elementos

Hay diversos criterios para ajustar el espacio entre las vistas encadenadas:



Se debe añadir a la primera View que forma la cadena el siguiente atributo:

app:layout_constraint<Horizontal/Vertical>_chainStyle: <estilo>

Nota*: *weighted* no es un tipo de cadena. Para lograr ese comportamiento hay que añadir pesos a cada *View* y poner su magnitud correspondiente en *Odp*.

Ejemplo de cadena en ConstraintLayout

```
<androidx.constraintlayout.widget.ConstraintLayout  
... >
```

activity_main.xml

```
<Button
```

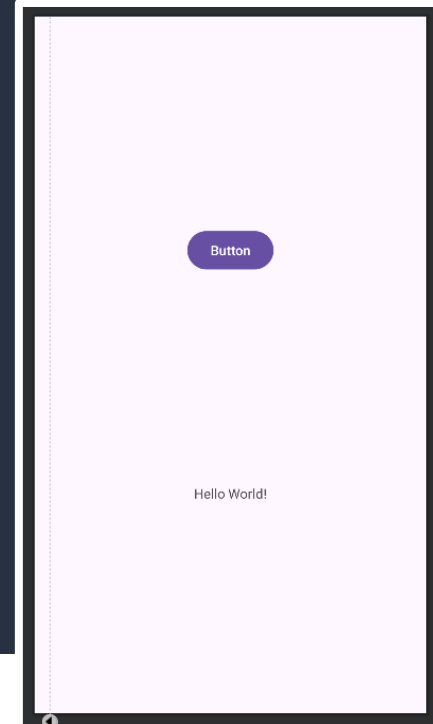
```
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:layout_constraintBottom_toTopOf="@+id/textView"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintStart_toStartOf="parent"  
    app:layout_constraintTop_toTopOf="parent"  
    app:layout_constraintVertical_chainStyle="spread" />
```

Restricciones
complementarias
forman una cadena

```
<TextView
```

```
    android:id="@+id/textView"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:layout_constraintTop_toBottomOf="@+id/button"  
    app:layout_constraintBottom_toBottomOf="parent"  
    app:layout_constraintEnd_toEndOf="parent"  
    app:layout_constraintStart_toStartOf="parent" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```



Ejercicio 1

Diseñar el layout que aparece en la figura de la derecha haciendo uso de los ViewGroup y elementos de la interfaz necesarios.

Recuerda que puedes anidar ConstraintLayout

Atributos de interés:

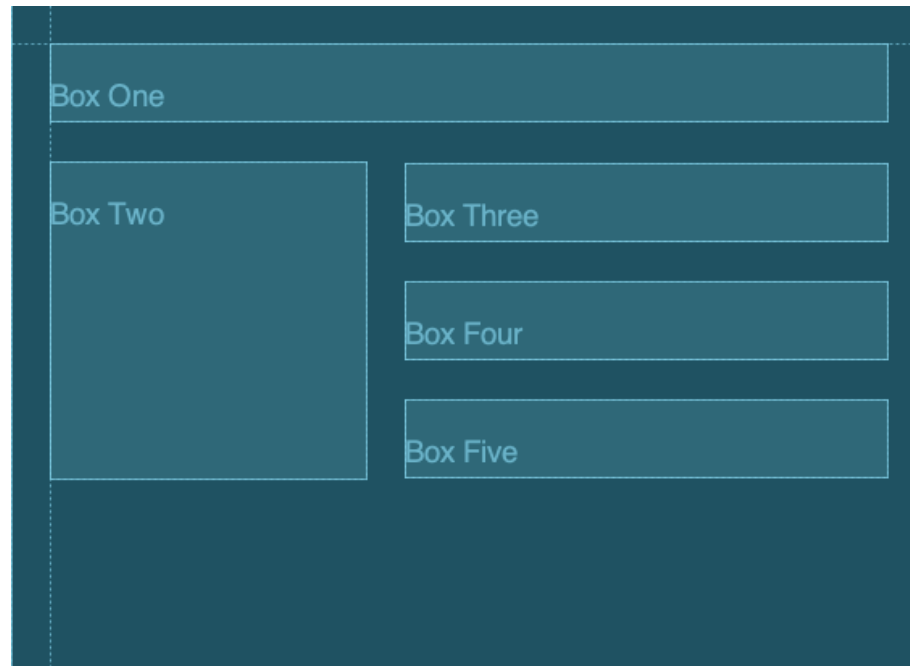
- **android:background** (color del fondo)
- **android:padding(X)**
- **android:layout_margin(X)**
- **app:layout_constraint(Vertical|Horizontal)_weight** (para las cadenas)



Mejorando el diseño

Guías

- Permite posicionar varias vistas en relación con una única guía
- Pueden ser verticales u horizontales
- Permiten una mayor colaboración con los equipos de diseño/UX
- No se dibujan en el dispositivo



Guías

```
<androidx.constraintlayout.widget.Guideline
```

activity_main.xml

```
    android:id="@+id/guideline"
```

```
    android:layout_width="wrap_content"
```

```
    android:layout_height="wrap_content"
```

```
    android:orientation="vertical"
```

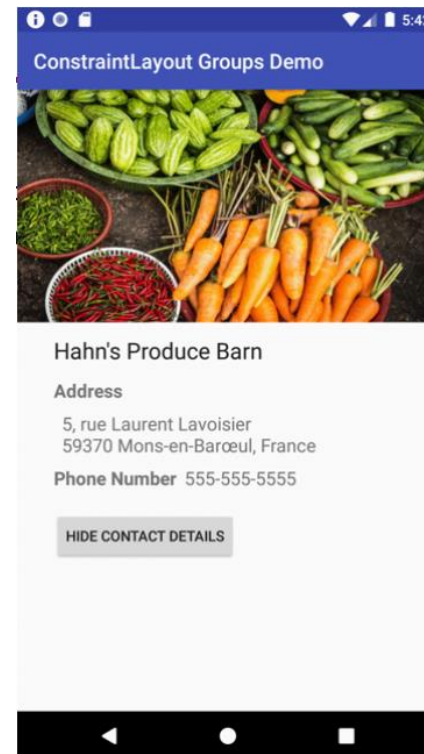
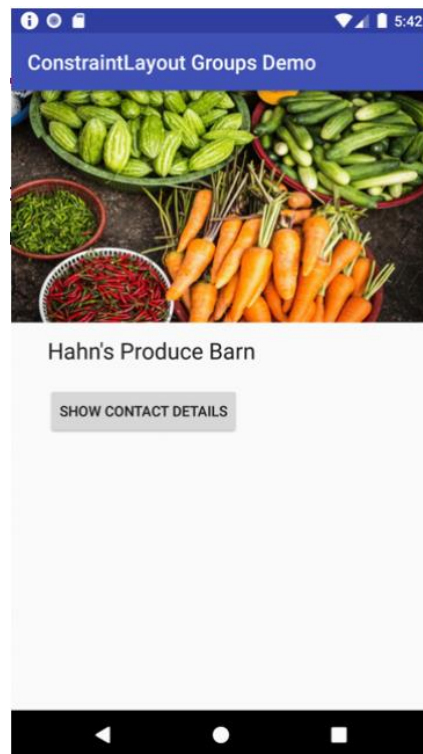
```
    app:layout_constraintGuide_begin="16dp" />
```

Atributos para controlarlas

- `android:orientation`
- `layout_constraintGuide_begin`
- `layout_constraintGuide_end`
- `layout_constraintGuide_percent`

Agrupamiento de elementos

- Los *Group* permite “asociar” elementos de la interfaz creando un agrupamiento
- La visibilidad del agrupamiento se puede controlar y ocultar y mostrar todos los elementos a la vez



Ejemplo de agrupamiento

1. En el XML se crea un Group que asocia diferentes vistas (*referenced_ids*)
2. En el .kt se puede modificar su atributo *visibility* (VISIBLE | GONE)

```
class MainActivity : AppCompatActivity() {  
    private lateinit var grupo: Group  
    private lateinit var boton: Button  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        boton = findViewById(R.id.boton)  
        grupo = findViewById(R.id.grupo)  
        boton.setOnClickListener {  
            if (grupo.visibility == View.VISIBLE){  
                grupo.visibility = View.GONE  
            }else{  
                grupo.visibility = View.VISIBLE  
            }  
        }  
    }  
}
```

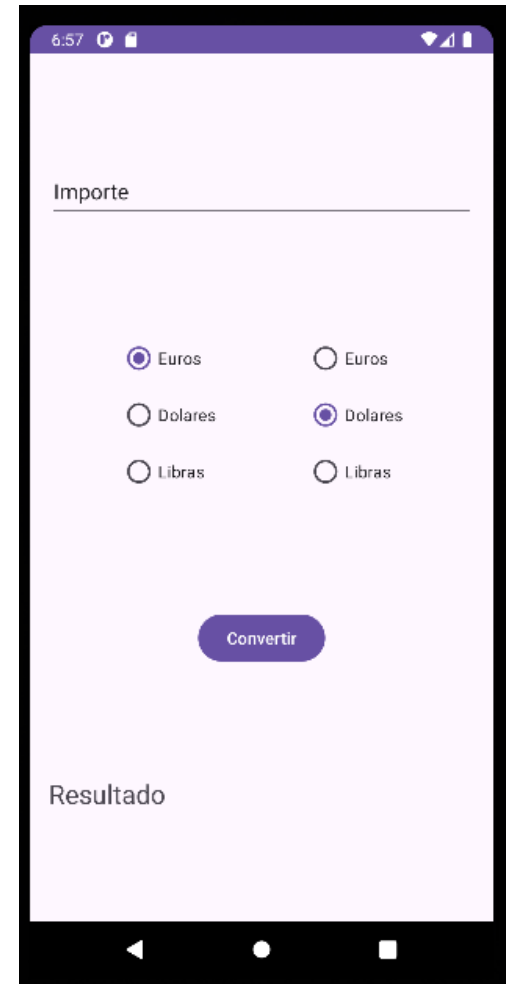
MainActivity.kt

```
<androidx.constraintlayout.widget.Group  
    android:id="@+id/grupo"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content"  
    app:constraint_referenced_ids="boton1,textview2"/>
```

activity_main.xml

Ejercicio 2

1. Diseña la aplicación que aparece en la figura de la derecha haciendo uso de los ViewGroup y elementos de la interfaz necesarios.
2. Añade la funcionalidad necesaria para que al pulsar el botón Convertir, se calcule el cambio de divisa para el importe introducido en el campo de texto de arriba teniendo en cuenta la divisa de origen (izquierda) y la de destino (derecha).
3. Modifica los atributos del campo de texto de arriba para permitir al usuario introducir únicamente números decimales.



Analizando nuestra interfaz

Análisis de accesibilidad

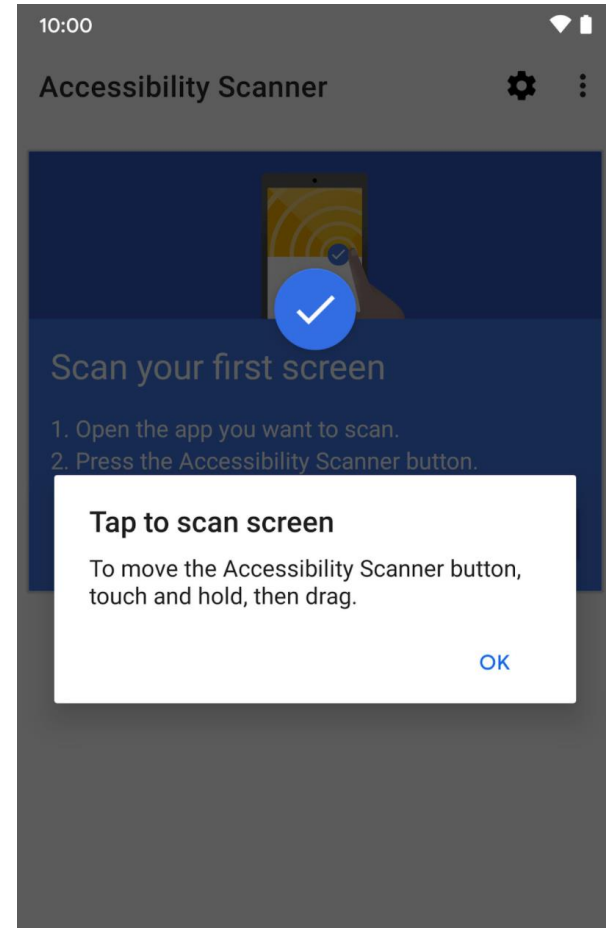
La accesibilidad busca la **mejora del diseño** y la funcionalidad de una aplicación **para facilitar su uso a más personas**, incluidas aquellas con discapacidades.

Hacer una app más accesible mejora la experiencia y **beneficia a todos los usuarios**.

Test de accesibilidad / Accessibility Scanner

App que escanea tu pantalla y sugiere mejoras para hacer tu aplicación más accesible:

- Tamaños de objetivos táctiles
- Vistas en las que se puede hacer clic
- Contraste de texto e imagen



Ejercicio 3

1. Instala la aplicación Test de accesibilidad desde la Play Store
2. Sigue los pasos indicados para habilitar el análisis de la pantalla
3. Abre cualquier aplicación que tengas instalada, pásale el escáner y explora las recomendaciones

Test de Accesibilidad

Google LLC

4,2★
12,4 mil reseñas

1 M+
Descargas

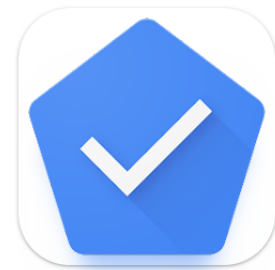
3
PEGI 3

Descargar

Compartir

Añadir a la lista de deseos

Esta aplicación está disponible para todos tus dispositivos



¿Te interesa?

Android tiene más opciones de accesibilidad como *TalkBack* o la navegación por switches

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Actividades y ciclo de vida



Universidad
Rey Juan Carlos

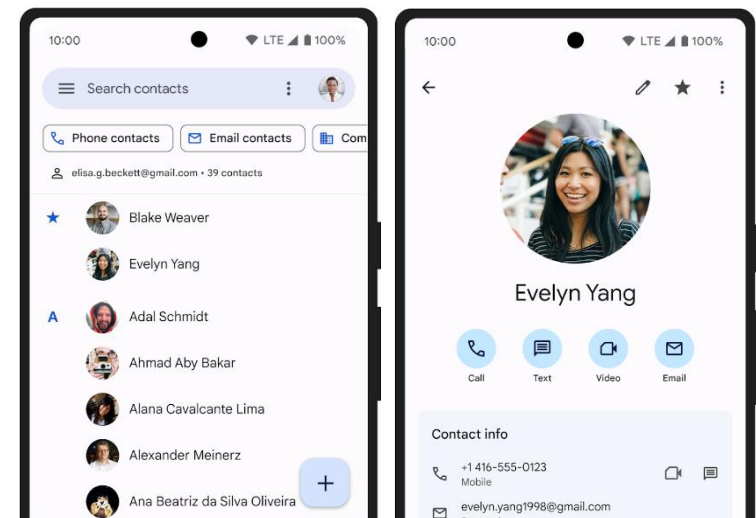
Actividades en Android

La Actividad es un componente fundamental de las aplicaciones Android. **Una actividad proporciona la ventana en la que la app dibuja su IU y con la que el usuario interactúa.** Esta ventana suele ocupar toda la pantalla, aunque puede ser más pequeña que ella y flotar sobre otras ventanas.

Por lo general, una actividad representa una pantalla en una aplicación, y se compone de una **parte gráfica** (xml/compose) y de **una lógica** (kotlin/java).

Una actividad de agenda tendría:

- Una actividad para la lista de contactos
- Una actividad para mostrar un contacto
- Una actividad para crear un contacto
- Una actividad para las preferencias



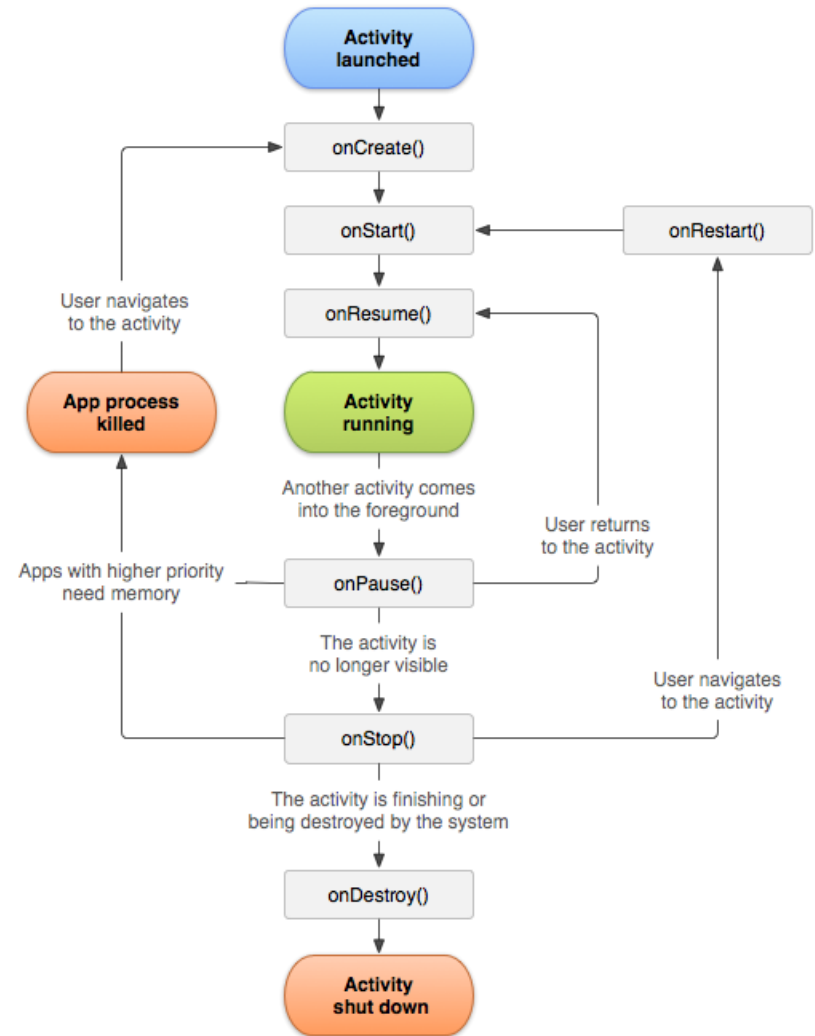
Ciclo de vida de una actividad

Las actividades de una app pasan por diferentes estados de su ciclo de vida.

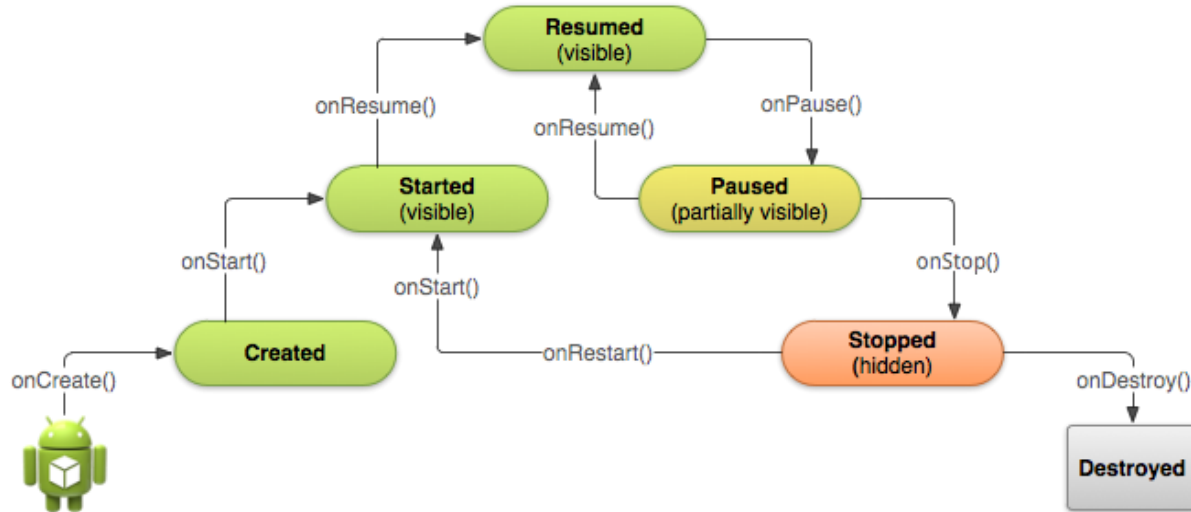
La clase Activity proporciona una serie de **callbacks** que le **informan a la actividad cuándo cambia un estado**.

Estos *callbacks* se pueden sobrescribir para evitar fugas de memoria y fallos de la aplicación y conservar los datos y el estado del usuario si, por ejemplo:

- El usuario abandona temporalmente la aplicación y luego regresa.
- El usuario es interrumpido por una llamada telefónica.
- El usuario gira el dispositivo.



Callbacks y estados de una actividad



Callbacks	Estado al que pasa	Descripción
<code>onCreate()</code>	Created	La actividad está siendo inicializada
<code>onStart()</code>	Started	La actividad es visible al usuario
<code>onResume()</code>	Resumed	La actividad tiene el foco de entrada
<code>onPause()</code>	Paused	La actividad no tiene el foco
<code>onStop()</code>	Stopped	La actividad no está visible
<code>onDestroy()</code>	Destroyed	La actividad se ha destruido

Callbacks del ciclo de vida

onCreate(): se llama al crear la actividad. Es donde se prepara la interfaz gráfica de la pantalla. Aquí se realiza la lógica de inicio básica de la aplicación, que ocurre una sola vez en toda el ciclo de vida. El siguiente método es onStart().

onStart(): se ejecuta justo antes de que la aplicación sea visible al usuario. El siguiente método de ciclo de vida llamado será onStop() u onResume().

onResume(): se ejecuta en el momento en que la actividad se encuentra en la parte superior de la pila, justo antes de que el usuario pueda interactuar con ella. El siguiente método será onPause().

onPause(): se llama cuando la actividad deja de estar en primer plano, aunque seguirá visible si el usuario está en el modo multiventana. Aquí debemos aprovechar para pausar o ajustar las operaciones que no sean necesarias mientras la actividad está pausada. También se pueden liberar recursos del sistema, sensores, etc. El método siguiente será onResume() u onStop().

Callbacks del ciclo de vida (*cont*)

onStop(): se ejecuta cuando la actividad se hace invisible al usuario. Puede ser porque otra actividad la tape o porque vaya a ser destruida. Se usa para liberar o ajustar los recursos que no se necesitan mientras la app no es visible para el usuario. El siguiente método puede ser `onRestart()` u `onDestroy()`.

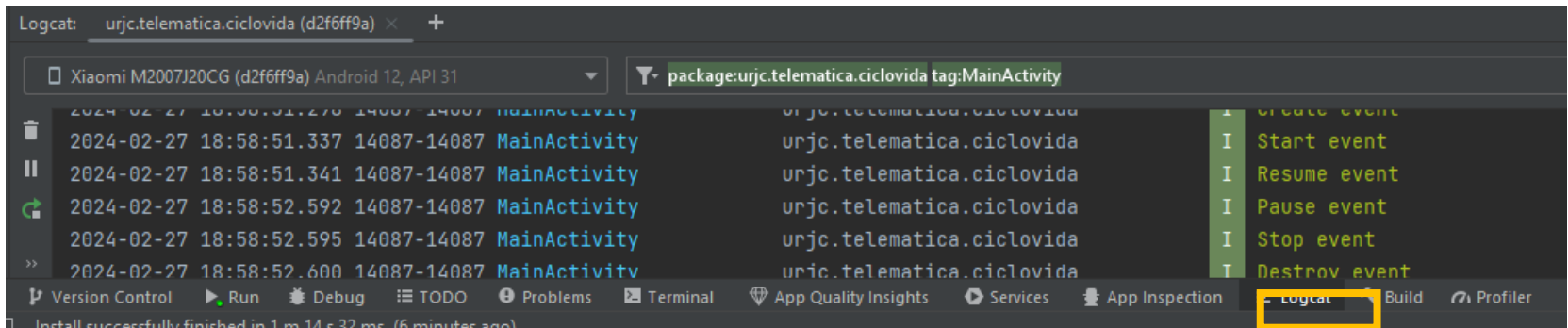
onDestroy(): se llama antes de destruir la actividad. Durante la destrucción se pierden todos los datos asociados a ella, por lo que puede aprovecharse para controlar la persistencia de datos. Se da cuando se ejecuta *finish()* o porque el sistema elimina la actividad para conseguir más recursos.

onRestart(): se llama cuando una actividad que se había parado vuelve a estar activa, justo antes de que comience de nuevo. Tras ella se llama a `onStart()`.

Logging

Clase Log

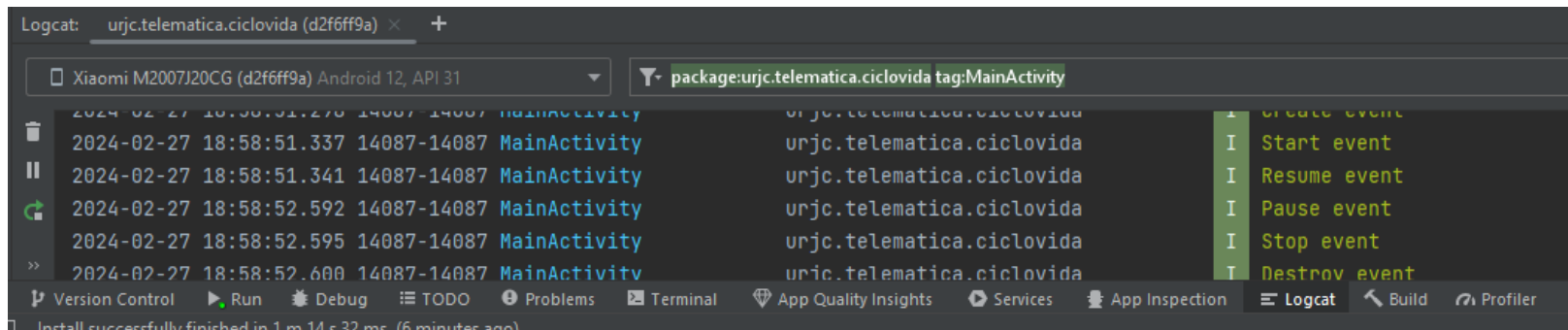
Monitorización de eventos o el estado de la app. Ejemplo: `Log.d("TAG", "Mensaje")`



Nivel de prioridad	Función de Log
Verbose	<i>Log.v(String, String)</i>
Debug	<i>Log.d(String, String)</i>
Info	<i>Log.i(String, String)</i>
Warning	<i>Log.w(String, String)</i>
Error	<i>Log.e(String, String)</i>

Ejercicio 1

1. Crea una aplicación nueva
2. En la actividad principal, sobrescribe todas las callback de cambio de estado de la actividad y añade un mensaje de log a cada una de ellas.
3. Lanza la aplicación en tu móvil / simulador y observa en el *Logcat* cuando se producen las llamadas a cada callback.



Intents

Múltiples actividades

Como hemos visto, a veces **la funcionalidad de la aplicación puede estar separada en varias pantallas.**

Ejemplos:

- Ver los detalles de un único elemento (una peli en una plataforma)
- Crear un nuevo elemento (por ejemplo, un nuevo correo electrónico)
- Mostrar la configuración de una aplicación
- Acceder a servicios de otras aplicaciones (por ejemplo, galería de fotos o examinar documentos)

Intents

Android utiliza una construcción llamada *Intent* para **solicitar una acción de otro componente**.

Un *Intent* puede utilizarse para especificar una **petición de transición** a otra Actividad. Sin embargo, también **puede contener datos** para que la Actividad de destino los utilice (como detalles sobre un ítem a ser mostrado), así como emplearse para **pasar datos de vuelta** a la actividad de origen.

Una *Intent* suele contener dos datos principales:

- Acción a realizar (por ejemplo, ACTION_VIEW, ACTION_EDIT, ACTION_MAIN)
- Datos sobre los que operar (por ejemplo, el registro de una persona en la base de datos de contactos)

Intent implícito

Los Intent implícitos **no especifican un objetivo previsto**, sino que proporcionan información suficiente para que **el sistema resuelva qué componente debe gestionar la llamada**.

- Proporciona una acción genérica que la aplicación puede realizar
- Se resuelve mediante la asignación del tipo de datos y la acción a componentes conocidos.
- Permite que cualquier aplicación que cumpla los criterios gestione la solicitud.

```
fun enviarEmail() {  
    val intent = Intent(Intent.ACTION_SEND)  
    intent.type = "text/plain"  
    intent.putExtra(Intent.EXTRA_EMAIL, "hola@ejemplo.es")  
    intent.putExtra(Intent.EXTRA_TEXT, "Como estas?")  
  
    if (intent.resolveActivity(packageManager) != null) {  
        startActivity(intent)  
    }  
}
```



Comprueba que existe una app capaz de gestionar el *Intent* lanzado

Ejercicio 2

1. Crea una aplicación que tenga un campo de texto y un botón.
2. El campo de texto mostrará un *hint* indicando que se debe introducir una URL
3. Al pulsar el botón, la aplicación deberá tomar la URL del cuadro y abrirla a través de un navegador externo (instalado ya en el teléfono).

El Intent deberá crearse utilizando su constructor que recibe:

- Una acción: *Intent.ACTION_VIEW*
- Datos: *Uri.parse(String)* [tipo URI]

Intent explícito

Los Intents explícitos son más estrictos, ya que **indican un componente específico que debe gestionar la solicitud**.

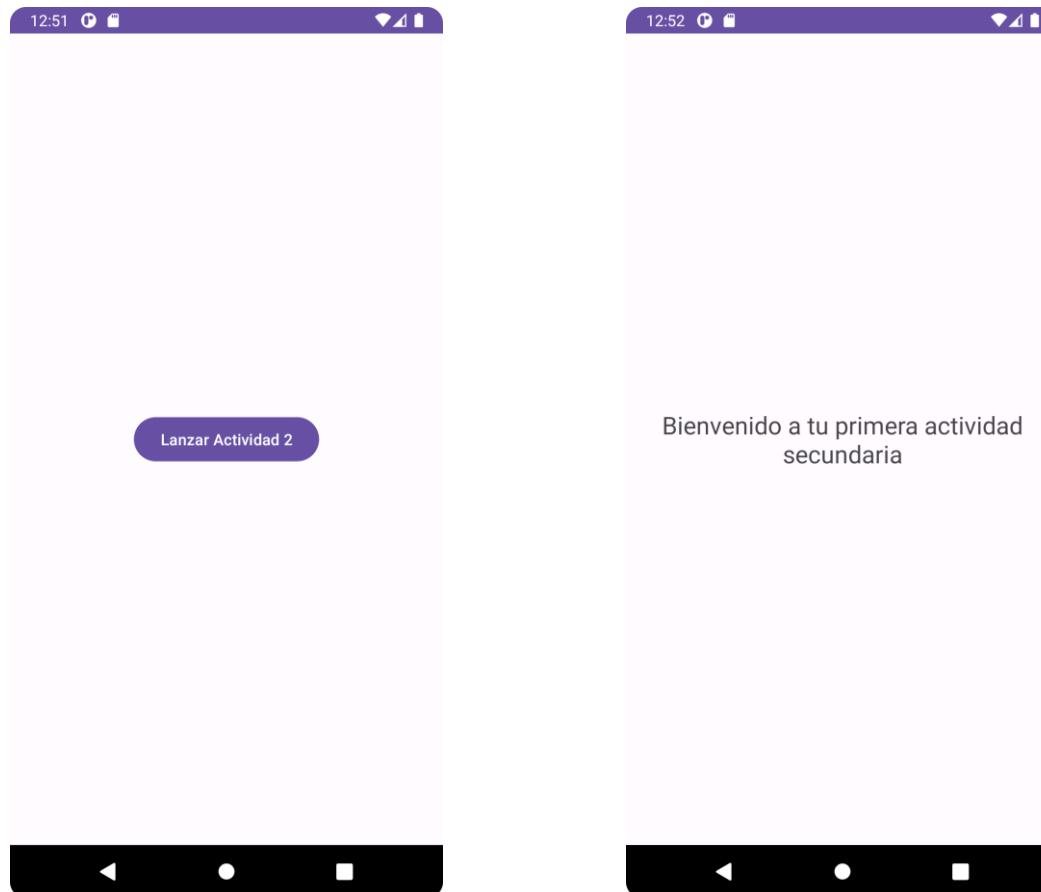
Se utilizan normalmente cuando se **navega entre componentes dentro de la aplicación** y se conoce el nombre de la clase. También puede utilizarlas para navegar a una aplicación conocida de terceros si está absolutamente seguro del tipo de Intent que pueden manejar.

```
fun verDetallesPelicula() {  
    val intent = Intent(this, PeliculaActivity::class.java)  
    intent.putExtra(PELICULA_ID, peli_id)  
    startActivity(intent)  
}
```

La función *putExtra()* nos permite enviar datos en el Intent que pueden ser utilizados en la actividad de destino.

Ejercicio 3

Crea una aplicación con un botón. Al hacer click en este se deberá lanzar una actividad secundaria como se muestra a continuación.



Recoger datos del Intent

Cuando se lanza una actividad a raíz de una interacción del usuario con una vista de nuestra aplicación (ej. un botón) **se pueden enviar datos a través del Intent**.

Para recoger estos datos en la actividad que se crea, se debe acceder a los datos “extra” recibidos en el elemento ***intent*** de la actividad.

```
class SecondaryActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_secondary)  
        val nombre = intent.getStringExtra("NOMBRE_PELICULA")  
        val visionada = intent.getBooleanExtra("VISTA", false)  
        // Recibir otros datos adicionales...  
    }  
}
```

Ejercicio 4

Partiendo de la aplicación desarrollada en el ejercicio anterior:

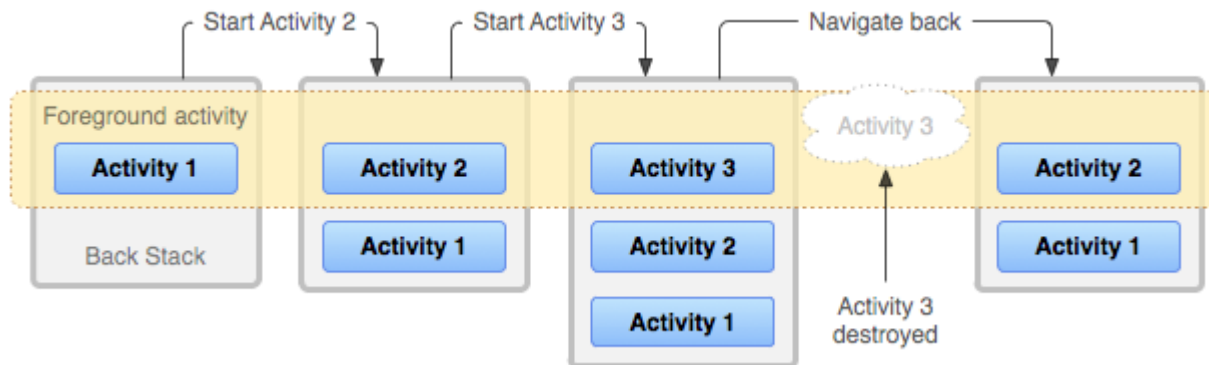
1. Añade un cuadro de texto a la actividad principal
2. Modifica el funcionamiento del botón para que, al pulsarlo, el texto contenido en el cuadro de texto se envíe a la actividad secundaria al lanzarla
3. Añade un campo de texto a la actividad secundaria
4. Modifica la actividad secundaria para mostrar el texto recibido en el campo de texto añadido

Pila de actividades

Pila de actividades o *back stack*

En Android, **las actividades se organizan en una pila (*back stack*)** en el orden en que se abre cada actividad. Es decir, **la última actividad en entrar es la primera en salir (*LIFO: last in, first out*)**.

Ejemplo: en la aplicación de contactos del móvil, la actividad principal muestra la lista de contactos guardados. Cuando pinchamos en uno, se abre otra actividad para ver los detalles. Esta nueva actividad se agrega a la pila. Al navegar hacia atrás, se destruye y se quita de la pila.



Sobrescribiendo la navegación hacia atrás

En ocasiones nos puede interesar **controlar lo que pasa cuando el usuario presiona el botón *Atrás***. Para ello, debemos sobrescribir su funcionamiento por defecto, asociando una *callback* personalizada con la lógica deseada.

```
MyActivity.kt

class MyActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_about)

        // ...
        onBackPressedDispatcher.addCallback(this, object : OnBackPressedCallback(true){
            override fun handleOnBackPressed() {
                TODO("Lógica a realizar")
                finish() // Si queremos cerrar la actividad
            }
        })
    }
}
```

Ejercicio 5

Modifica la aplicación desarrollada en el ejercicio anterior para que cuando el usuario esté en la actividad principal, deba pulsar dos veces el botón atrás antes de salir de la aplicación.

Al pulsar la primera vez, se mostrará un aviso (Toast): *Pulsa una vez más para cerrar la aplicación*

Uso de avisos flotantes Toast

```
Toast.makeText(this | applicationContext,"Mensaje", Toast.LENGTH_SHORT).show()
```

Desde un callback
de la Actividad

Desde un callback
de otro objeto

También se puede
Toast.LENGTH_LONG

**Lanzando actividades
para obtener un
resultado**

Actividades con resultado

Dependiendo de la naturaleza de la aplicación, **podemos tener tareas que deban realizarse en actividades distintas, devolviendo un resultado**. Cuando esto ocurre, la actividad secundaria se debe lanzar de una manera especial para poder capturar su resultado.

Ejemplo: en la aplicación de contactos del móvil, la creación de un contacto nuevo se suele manejar en una actividad independiente. Cuando el usuario introduce los datos o cancela la operación, se vuelve a la actividad principal y se actualiza el listado si es necesario.

Paso 1: lanzador de actividades con callback para manejar el resultado

MainActivity.kt

```
class MainActivity : AppCompatActivity() {
    private val getResult = registerForActivityResult(ActivityResultContracts.StartActivityForResult()){
        if (it.resultCode == Activity.RESULT_OK){
            Log.i(TAG, "Recibido ${it.data?.getStringExtra("nombre")}")
        }else{
            Log.i(TAG, "Operación cancelada")
        }
    }
}
```


Actividades con resultado (*cont*)

Paso 2: lanzar el intent de la actividad con el lanzador creado

MainActivity.kt

```
class MainActivity : AppCompatActivity() {  
    private lateinit var btnAcerca : Button  
    // ...  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        btnAcerca = findViewById(R.id.btnAbout)  
        btnAcerca.setOnClickListener {  
            val intent = Intent(this, SecondaryActivity::class.java)  
            getResult.launch(intent) // En sustitución de: startActivity(intent)  
        }  
    }  
}
```

Paso 3: devolver un resultado desde la actividad lanzada

SecondaryActivity.kt

```
class SecondaryActivity : AppCompatActivity() {  
    // ...  
    setResult(Activity.RESULT_OK, Intent().putExtra("nombre", "Javier"))  
    // setResult(Activity.RESULT_CANCELED)  
    finish()  
}
```

Ejercicio 6

Crea una aplicación con dos botones, uno al lado del otro, situados en la parte inferior de la pantalla. Añade un TextView que ocupe el resto de la interfaz.

El botón derecho deberá lanzar una actividad secundaria que permita al usuario, mediante los campos pertinentes, enviar dos valores de vuelta: el nombre de un producto y la cantidad a comprar.

Cuando finalice la operación con éxito, la actividad principal mostrará los datos recibidos en una línea nueva del TextView. Si, por el contrario, el usuario navega hacia atrás, se mostrará un aviso flotante (Toast) indicando que la operación se ha cancelado.

El botón izquierdo servirá para limpiar la lista de la compra y resetear el TextView.

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

RecyclerView y Adapters



Universidad
Rey Juan Carlos

RecyclerView

Cuando el contenido de tu diseño no sea fijo (por ejemplo, tu lista de contactos), se puede usar un *RecyclerView*. Este *ViewGroup* facilita la gestión de vistas que se modifican durante el tiempo de ejecución.

RecyclerView permite mostrar de manera eficiente grandes conjuntos de datos. Tú proporcionas los **datos** y defines el **aspecto** de cada elemento, y la biblioteca *RecyclerView* creará los **elementos de forma dinámica** cuando se los necesite.

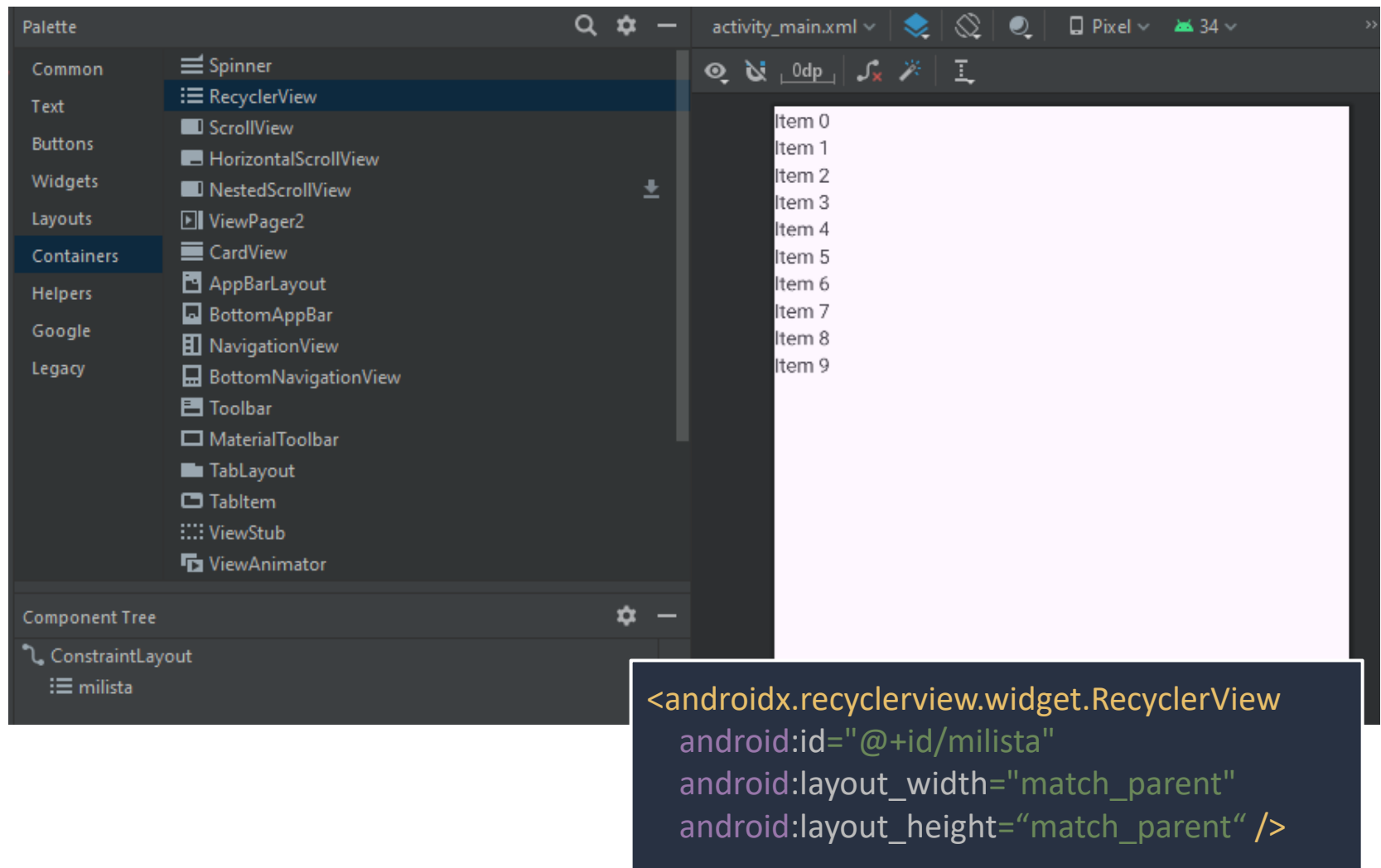


Componentes clave

Varias clases trabajan juntas para compilar tu lista dinámica.

- **RecyclerView** es el ViewGroup que contiene las vistas correspondientes a tus datos. Se puede agregar al layout desde el editor de diseños (*Containers*).
- **Contenedor de vistas:** representa el diseño de cada elemento que se va a mostrar. Contiene atributos que referencian todas las vistas que componen de su layout. Para definirlo: extiende *RecyclerView.ViewHolder*.
- **Adaptador:** es el encargado de crear las vistas de cada elemento y rellenarlo con los datos correspondientes. Para definirlo: extiende *RecyclerView.Adapter*.
- **Administrador de diseño:** organiza los elementos de tu lista. Se puede utilizar *LinearLayoutManager*, *GridLayoutManager*, o definir uno propio.

Añadir el RecyclerView al diseño



Definir el diseño de cada item

En un archivo .xml aparte dentro de *res/layout*, se diseña el aspecto de la vista que tendrá cada elemento de nuestra lista/cuadrícula.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="TextView"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Item_lista.xml

Crear el contenedor de vistas (ViewHolder)

Es una clase que se utiliza para representar y manipular la vista de cada elemento que se va a mostrar.

- Hereda de *RecyclerView.ViewHolder*
- Recibe una *View* en su constructor, que es la que contiene el diseño del elemento.
- Debe definir un atributo para cada vista del diseño, para posteriormente permitir al adaptador mostrar los datos correspondientes.

```
class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row) {  
    val textView = row.findViewById<TextView>(R.id.textView)  
}
```

MyViewHolder.kt

Crear el adaptador

Suministra los datos y diseños que muestra el RecyclerView. Un Adapter personalizado extiende de *RecyclerView.Adapter* y sobrescribe estas tres funciones:

onCreateViewHolder(): RecyclerView llama a este método siempre que necesita crear una *ViewHolder* nueva. El método crea y, luego, inicializa la ViewHolder y su View asociada, pero no completa el contenido de la vista.

onBindViewHolder(): RecyclerView llama a este método para asociar una *ViewHolder* con los datos. El método recupera los datos correspondientes y los usa para completar el diseño del contenedor de vistas.

getItemCount(): RecyclerView llama a este método para obtener el tamaño del conjunto de datos. Se utiliza internamente para saber cuando no hay más elementos que se puedan mostrar.

Crear el adaptador (*cont.*)

Ejemplo de un Adapter propio cuya fuente de datos es una lista de String.

onCreateViewHolder crea la vista mediante el ViewHolder con el diseño de un elemento. *onBindViewHolder* rellena la vista con los datos de la posición correspondiente.

MyAdapter.kt

```
class MyAdapter(val data: List<String>) : RecyclerView.Adapter<MyViewHolder>() {  
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MyViewHolder {  
        val layout = LayoutInflater.from(parent.context).inflate(R.layout.item_lista,  
            parent, false)  
        return MyViewHolder(layout)  
    }  
  
    override fun onBindViewHolder(holder: MyViewHolder, position: Int) {  
        holder.textView.text = data[position]  
    }  
  
    override fun getItemCount(): Int = data.size  
}
```

Devuelve el número de
elementos

Configurar el RecyclerView en la actividad

1. Inicializar la variable que referencia al RecyclerView en el layout.
2. Establecer un LayoutManager en él para indicar como mostrar los elementos.
3. Instanciar el adaptador con los datos y asignárselo al RecyclerView.

```
class MainActivity : AppCompatActivity() {  
    private lateinit var miLista: RecyclerView  
    private val data = mutableListOf<String>()  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        miLista = findViewById(R.id.milista)  
        miLista.layoutManager = LinearLayoutManager(this)  
        // LinearLayoutManager(this).apply { orientation=LinearLayoutManager.HORIZONTAL }  
        var miAdaptador = MyAdapter(data)  
        miLista.adapter = miAdaptador  
    }  
}
```

MainActivity.kt

Ejercicio 1

1. Crea una aplicación nueva con una única actividad
2. En el diseño, incluye un botón y un RecyclerView que ocupe el resto de la pantalla
3. Crea las clases y diseños necesarios para mostrar elementos que contengan una String en mayúsculas y en negrita.
4. Añade la lógica necesaria para que al pulsar el botón se añada un nuevo elemento a la lista dinámica mostrada en el RecyclerView

Para refrescar el RecyclerView tras modificar los datos hay varias opciones

```
miAdaptador.notifyDataSetChanged() // No recomendada, reconstruye todo de cero  
miAdaptador.notifyItemInserted(data.size) // Más rápida. Específica para insertar
```

Otras: notifyItemChanged, notifyItemRemoved, notifyItemRangeChanged, notifyItemRangeInserted, notifyItemRangeRemoved

RecyclerView con datos propios

Clase modelo

Cuando queremos mostrar algo más que un dato simple por cada elemento de la lista, se deben **utilizar datos propios**. Para ello, creamos **una *clase modelo***.

Esta clase modelo, típicamente una *data class*, recoge todos los atributos del elemento que se quiere mostrar (y quizás algunos que no).

Ejemplo:

Para una aplicación donde queremos mostrar una lista de la compra, cada elemento puede contener el nombre del producto y las unidades a adquirir.

```
data class ItemCompra(val producto: String, val cantidad: Int) : Serializable
```

ItemCompra.kt



Permite meterlo como extra en un Intent.

Ejercicio 2

1. Crea una aplicación nueva con una única actividad para guardar contactos en una lista.
2. En el diseño, abajo, incluye dos campos de texto (para el nombre y el teléfono) y un botón. Además, añade un RecyclerView que ocupe el resto de la pantalla
3. Crea las clases y diseños necesarios para mostrar, para cada elemento, el nombre y el teléfono de un contacto.
4. Añade la lógica necesaria para que al pulsar el botón se añada un nuevo contacto a la lista dinámica mostrada en el RecyclerView

Eventos en un RecyclerView

Interfaz de evento en el adaptador

Varias clases trabajan juntas para compilar tu lista dinámica.

```
interface RecyclerViewEvent {  
    fun onItemClick(position: Int)  
}
```

1. Definimos una interfaz con un método por evento

MainActivity.kt

2. Añadimos la interfaz como atributo al constructor

```
class MyAdapter(val data: List<String>, val listener: RecyclerViewEvent? = null) :  
    RecyclerView.Adapter<MyAdapter.MyViewHolder>() {  
    inner class MyViewHolder(val row: View) : RecyclerView.ViewHolder(row), View.OnClickListener {  
        val textView = row.findViewById<TextView>(R.id.textView)  
        init {  
            row.setOnClickListener(this)  
        }  
  
        override fun onClick(p0: View?) {  
            val position = adapterPosition  
            if (position != RecyclerView.NO_POSITION) {  
                listener?.onItemClick(position)  
            }  
        }  
    }  
    // ... onCreateViewHolder y demás funciones  
}
```

3. El *ViewHolder* lo ponemos como una *inner class* que implementa *View.OnClickListener*

4. Asignamos el *callback* a la vista entera, o a uno de sus elementos

5. Implementamos el método *onClick*, donde llamamos a la función recibida por el constructor

Asignar función en la actividad

Al instanciar el Adapter, se puede pasar un segundo parámetro opcional al constructor con un objeto que implemente la interfaz definida para asignarle una función al evento *onClick* del elemento.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
    miLista = findViewById(R.id.milista)  
    miLista.layoutManager = LinearLayoutManager(this)  
    var miAdaptador = MyAdapter(data, object : RecyclerViewEvent {  
        override fun onItemClick(position: Int) {  
            TODO("Implementar la funcion a ejecutar cuando se haga click")  
        }  
    })  
    miLista.adapter = miAdaptador  
}
```

MainActivity.kt

Ejercicio 3

Siguiendo con el ejercicio anterior de la lista de contactos:

1. Añade dos campos de entrada más al cuadro inferior, uno para recoger el email y otro para la fecha de cumpleaños.
2. Modifica la clase modelo para que almacene todos los atributos de un contacto.
3. Al pulsar el botón, se añadirá el contacto a la lista. El diseño de la vista de cada elemento en la lista no varía (muestra solo nombre y teléfono).
4. Crea una segunda actividad *DetallesContactoActivity* donde se muestren todos los atributos de un contacto (nombre, número, email y cumpleaños).
5. Modifica la aplicación para que, al hacer click en un contacto de la lista, se lance la actividad con la información detallada del mismo.

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

AppBar, NavDrawer y FAB



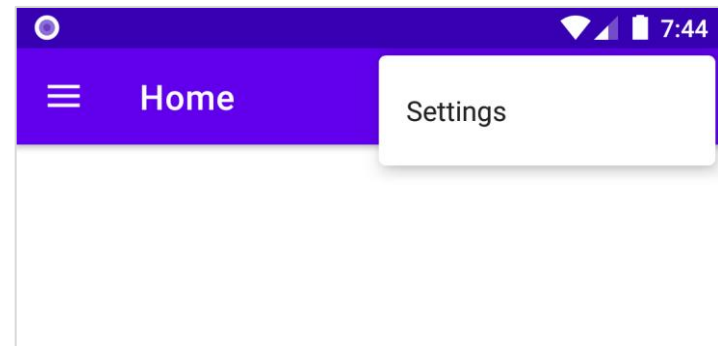
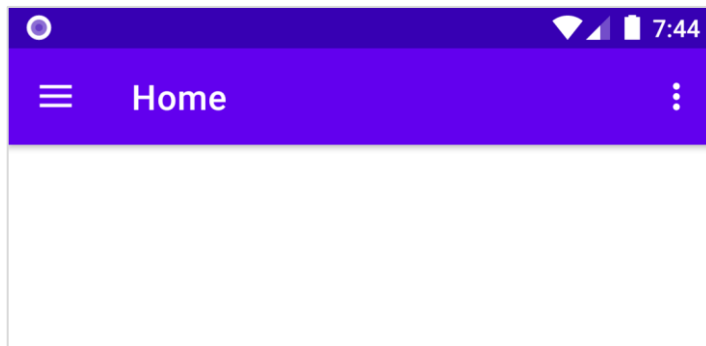
Universidad
Rey Juan Carlos

**Barra superior
(Appbar)**

Barra de la app

La barra superior o *barra de acciones* (*ActionBar*) proporciona una estructura visual coherente con otras apps, haciendo comprensible su manejo:

- Proporciona una identidad a tu app e indica la ubicación del usuario en ella.
- Ofrece acceso predecible a acciones importantes, como la búsqueda.
- Compatible con la navegación mediante botones y menús.



Definir un menú

Crear un nuevo .xml de tipo Menu desde *res/* que contenga los elementos de nuestro menu.

- Definir para cada item: id, icono y título.
- Elegir cuáles mostrar en función del espacio ajustando su *app:showAsAction*

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
      xmlns:app="http://schemas.android.com/apk/res-auto">

  <item
    android:id="@+id/action_gallery"
    android:icon="@android:drawable/ic_menu_gallery"
    android:title="@string/menu_gallery"
    app:showAsAction="ifRoom"/>
  <item
    android:id="@+id/action_settings"
    android:orderInCategory="100"
    android:title="@string/action_settings"
    android:icon="@android:drawable/ic_menu_preferences"
    app:showAsAction="never" />
</menu>
```

menu_toolbar.xml

Añadir la *appbar* al diseño de la actividad

Para añadir la barra superior de navegación a la actividad, en el .xml correspondiente se añade un *AppBarLayout* que contenga una *MaterialToolbar*.

Además, hay que asegurarse de que el *Theme* de la aplicación definido en el *AndroidManifest.xml* termina en *NoActionBar*.

```
<com.google.android.material.appbar.AppBarLayout
    android:id="@+id/appbarlayout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:layout_constraintTop_toTopOf="parent">

    <com.google.android.material.appbar.MaterialToolbar
        android:id="@+id/toolbar"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:title="Home"
        app:navigationIcon="@drawable/ic_menu"
        style="@style/Widget.MaterialComponents.Toolbar.Primary" />

</com.google.android.material.appbar.AppBarLayout>
```

activity_main.kt

Icono opcional. Si no se especifica, se pone la flecha de ir atrás

Inicializar la *appbar* en la actividad

Como con el resto de elementos de diseño que queremos gestionar, en la actividad se debe crear un atributo y asociarlo a su vista. Además:

1. Asignar la *toolbar* como barra de acción via *setSupportActionBar()*.
2. Sobreescibir *onCreateOptionsMenu* para inflar el menú antes creado.

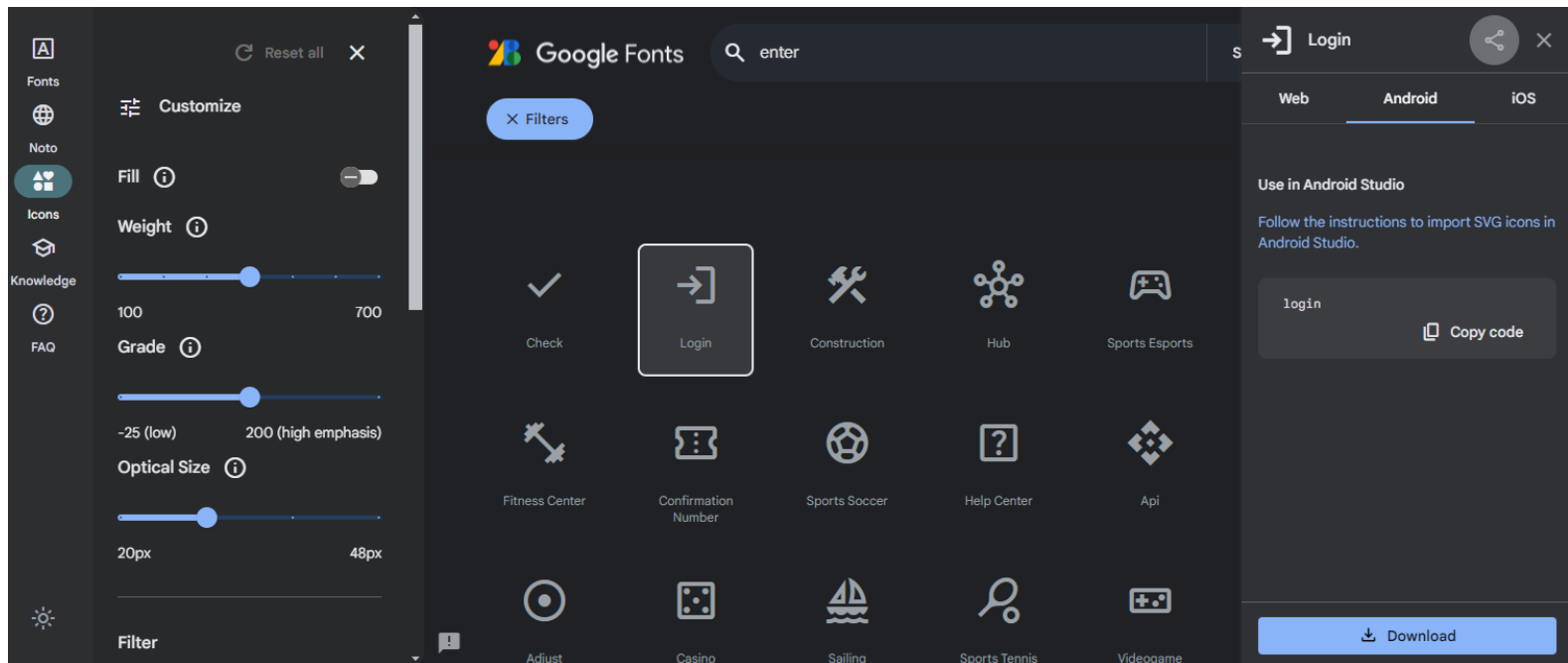
```
class MainActivity : AppCompatActivity() {  
    private lateinit var toolbar: Toolbar  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
        toolbar = findViewById(R.id.toolbar)  
        setSupportActionBar(toolbar)  
    }  
  
    override fun onCreateOptionsMenu(menu: Menu?): Boolean {  
        menuInflater.inflate(R.menu.menu_toolbar, menu)  
        return true  
    }  
}
```

MainActivity.kt

Iconos de *Material Design*

Material Design es la guía de diseño estándar en Android. Para usar sus iconos:

1. Ir a <https://fonts.google.com/icons?icon.platform=android>
2. Buscar y seleccionar el icono deseado
3. Descargar el archivo .xml de Android y pegarlo en *res/drawable/*



Responder a interacciones

Para capturar las interacciones con los elementos del menú de la barra superior se debe sobrescribir el método *onOptionsItemSelected*.

Recibe como parámetro un MenuItem, cuyo atributo itemId nos permite saber qué opción del menú ha sido clickada por el usuario.

```
class MainActivity : AppCompatActivity() {  
    override fun onOptionsItemSelected(item: MenuItem): Boolean {  
        when(item.itemId){  
            R.id.action_checkout -> Log.d("Toolbar", "Acción de la galería")  
            R.id.action_account -> Log.d("Toolbar", "Acción del carousel")  
            R.id.action_settings -> Log.d("Toolbar", "Acción de las preferencias")  
            else -> super.onOptionsItemSelected(item)  
        }  
        return true  
    }  
}
```

MainActivity.kt

Ejercicio 1

Siguiendo con el ejercicio de la app de contactos de la sesión anterior:

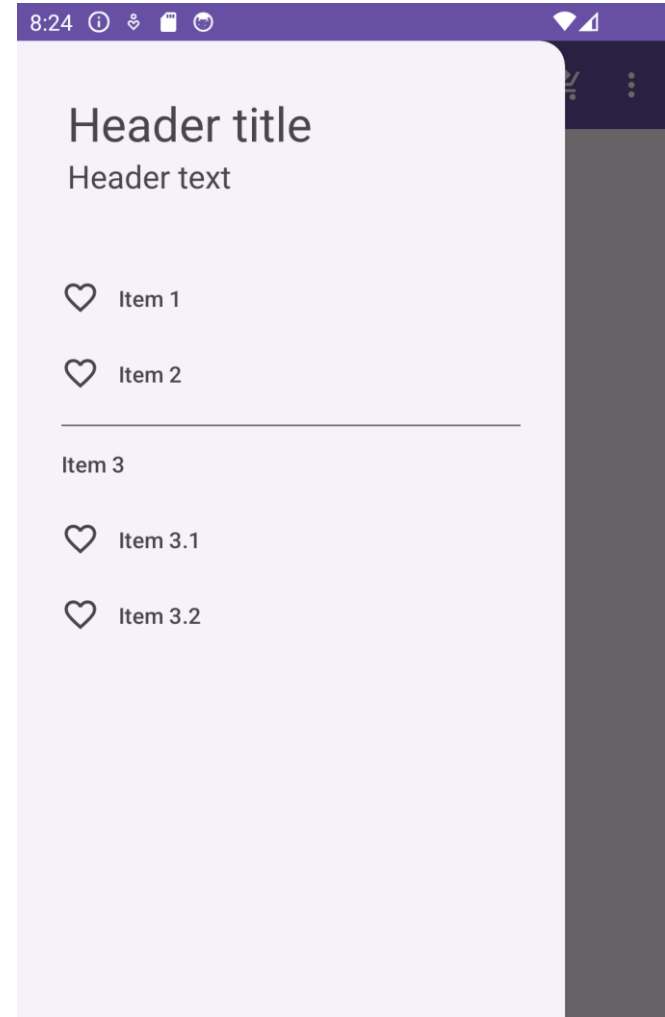
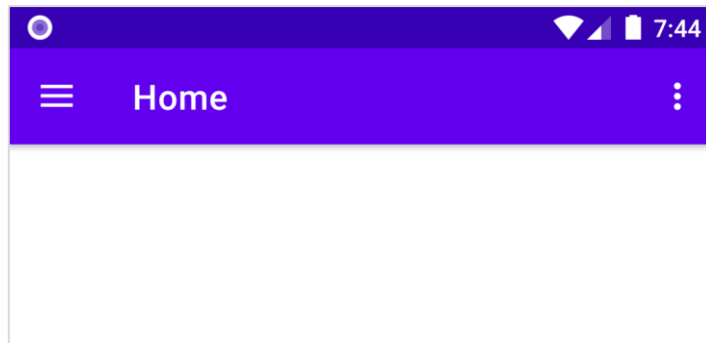
1. Realiza las modificaciones necesarias para añadir una barra de acciones a la aplicación.
2. Añade a la barra un menú con dos acciones:
 1. La primera: con el icono de Delete Sweep de la colección Material, que vacíe la lista de contactos. El icono se debe mostrar en la barra.
 2. La segunda: con el icono Info, que esté escondida en el menú desplegable, que muestre un Toast con el nombre del autor de la aplicación.

Menú lateral (Navigation Drawer)

Menú lateral

En Android, es común que las aplicaciones desplieguen un menú lateral que nos permita navegar a otras pantallas, gestionar nuestra cuenta o administrar las preferencias...

Ese menú se conoce como *Navigation Drawer* y está accesible desde el botón *Home* situado en la parte izquierda de la *toolbar*.



Definir un menú

menu_drawer.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
  <group android:checkableBehavior="single">
    <item
      android:id="@+id/nav_gallery"
      android:icon="@android:drawable/ic_menu_gallery"
      android:title="@string/menu_gallery"/>
    <item
      android:id="@+id/nav_slideshow"
      android:icon="@android:drawable/ic_menu_slideshow"
      android:title="@string/menu_slideshow"/>
  </group>
  <group
    android:id="@+id/group1"
    android:checkableBehavior="single">
    <item
      android:id="@+id/nav_settings"
      android:title="@string/action_settings"
      android:icon="@drawable/ic_settings"/>
  </group>
</menu>
```

Los grupos permiten crear agrupamientos entre opciones del menú relacionadas. Además, los grupos quedan separados visualmente por líneas divisoras

Añadir el menú lateral al diseño de la actividad

El diseño de una actividad con menú lateral debe tener un *DrawerLayout* como ViewGroup raíz. Dentro, , se añadirá un *NavigationView*, que será el elemento responsable de mostrar el menú.

Si queremos añadir más elementos de diseño, deberán ir en un *ConstraintLayout* al mismo nivel que el *NavigationView*.

```
<androidx.drawerlayout.widget.DrawerLayout
...
  android:id="@+id/drawer_layout"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  tools:openDrawer="start">

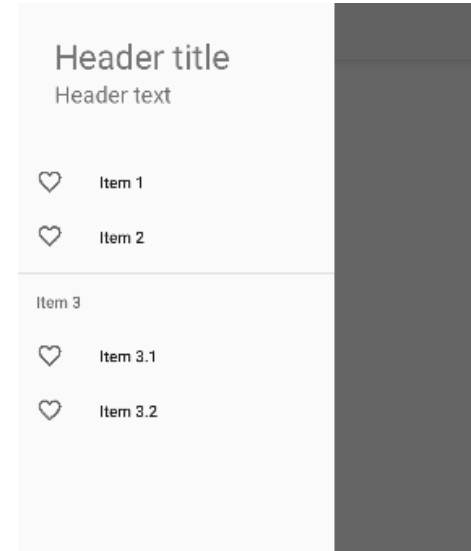
  <com.google.android.material.navigation.NavigationView
    android:id="@+id/nav_view"
    android:layout_width="wrap_content"
    android:layout_height="match_parent"
    android:layout_gravity="start"
    app:headerLayout="@layout/nav_header_main"
    app:menu="@menu/menu_drawer"/>
</androidx.drawerlayout.widget.DrawerLayout>
```

activity_main.xml

Reestructurar el diseño con *include*

En Android Studio es muy incómodo trabajar con los diseños basados en un `DrawerLayout`, puesto que el resto de elementos de la interfaz no se muestran en la vista previa.

Para solucionarlo, es recomendable mover el diseño de la actividad a otro `.xml`, e incluir este archivo al mismo nivel que el `NavigationView`.



```
<androidx.drawerlayout.widget.DrawerLayout
...
<include layout="@layout/content_main"/>

<com.google.android.material.navigation.NavigationView
... />
</androidx.drawerlayout.widget.DrawerLayout>
```

activity_main.xml

En `content_main.xml` se define el layout con los elementos de UI necesarios

Inicializar el menú en la actividad

Como con el resto de elementos de la UI, en la actividad creamos variables y las asociamos a sus vistas correspondientes. Además, mostramos en la appbar el botón de navegación hacia arriba llamando a *setDisplayHomeAsUpEnabled*.

Luego, configuramos la appbar para que el botón Home muestre el menú lateral.

```
class MainActivity : AppCompatActivity() {
    private lateinit var toolbar: Toolbar
    private lateinit var drawerLayout: DrawerLayout
    private lateinit var navView: NavigationView
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main_drawer)
        toolbar = findViewById(R.id.toolbar)
        setSupportActionBar(toolbar)
        supportActionBar?.setDisplayHomeAsUpEnabled(true)
        drawerLayout = findViewById(R.id.drawer_layout)
        navView = findViewById(R.id.nav_view)
    }
}
```

MainActivity.kt

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    when(item.itemId){
        android.R.id.home -> drawerLayout.open()
        // Resto de opciones del menú ...
    }
    return true
}
```

Responder a interacciones

Para asociar acciones a cada elemento del menú lateral, le asignamos un callback al `NavigationView` via `setNavigationItemSelectedListener`.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        // A continuación de lo anterior  
        navView.setNavigationItemSelectedListener {  
            drawerLayout.close()  
            when(it.itemId){  
                R.id.nav_gallery -> Log.d("Drawer", "Acción de la galería")  
                R.id.nav_slideshow -> Log.d("Drawer", "Acción del carrousel")  
                R.id.nav_settings -> Log.d("Drawer", "Acción de las preferencias")  
            }  
            true  
        }  
    }  
}
```

MainActivity.kt

Usando el objeto `DrawerLayout`
cerramos el menú

Ejercicio 2

Siguiendo con el ejercicio anterior:

1. Añade un menú lateral a la actividad principal que contenga dos grupos con dos y tres items, respectivamente.
2. Asocia un callback que muestre en el Logcat un mensaje diferente cuando se pulse cada uno de los elementos del menú.

**Botón de acción
flotante (FAB)**

Añadir un FAB a la actividad

Un botón de acción flotante (Floating Action Button) es un patrón de diseño muy común en Android.

Se trata de un botón circular en la esquina inferior derecha que se superpone a la vista de la actividad y que habilita la acción principal de la interfaz, que no se puede realizar mediante ningún otro elemento de la misma.

```
<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/fab"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintBottom_toBottomOf="parent"
    android:src="@drawable/ic_launcher_foreground"
    android:contentDescription="Añadir un contacto"
    android:layout_margin="16dp" />
```

content_main.xml

Asignar una acción al evento onClick

Una vez añadida la vista del FAB al diseño de la actividad, se debe implementar su funcionalidad en el *onCreate* de la Actividad correspondiente.

Se le asigna un *onClickListener* de la misma forma que con otros elementos.

MainActivity.kt

```
class MainActivity : AppCompatActivity() {  
    private lateinit var fab: FloatingActionButton  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        fab = findViewById(R.id.fab)  
        fab.setOnClickListener(){  
            Toast.makeText(this, "FAB", Toast.LENGTH_SHORT).show()  
        }  
    }  
}
```

Ejercicio 3

Siguiendo con el ejercicio anterior:

1. Añade un FAB a la actividad principal con el icono Material: *Person Add*.
2. Crea una nueva actividad que permita introducir la información de contacto que antes se captaba en la parte inferior de la actividad principal. Añade un botón abajo con el texto CREAR CONTACTO.
3. Modifica el diseño de la actividad principal para que el RecyclerView ocupe todo el espacio de la pantalla, quitando los campos de abajo.
4. Haz que el FAB lance la actividad para crear un contacto.
5. En la actividad de creación de contacto, implementa el evento *onClick* del botón para que al pulsarlo se envíen de vuelta a la actividad principal.
6. En la actividad principal, actualizar la lista con el nuevo contacto creado cuando se reciban los datos.

Referencias

- Curso *Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

DataBinding & ViewModel



Universidad
Rey Juan Carlos

DataBinding

findViewById

Para programar la lógica relacionada con la interfaz de nuestra aplicación necesitamos crear variables y asociarlas a su View mediante *findViewById*. Sin embargo, las IDs son globales a todas las pantallas, por lo que pueden aparecer errores de ejecución si se referencia a una vista que no está en el layout actual.

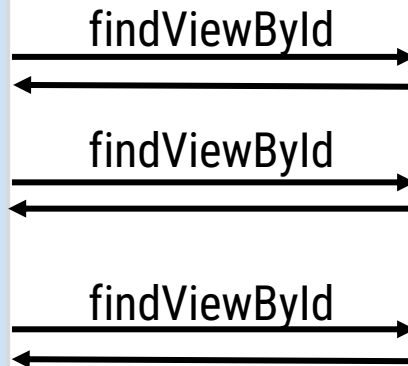
MainActivity.kt

```
val name = findViewById(...)
val age = findViewById(...)
val loc = findViewById(...)

name.text = ...
age.text = ...
loc.text = ...
```

activity_main.xml

```
<ConstraintLayout ... >
  <TextView
    android:id="@+id/name"/>
  <TextView
    android:id="@+id/age"/>
  <TextView
    android:id="@+id/loc"/>
</ConstraintLayout>
```



DataBinding

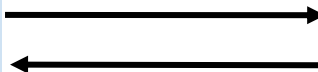
Como alternativa, se puede utilizar el enfoque *DataBinding* (vinculación de datos), que compila clases que enlazan directamente con las vistas del diseño actual en lugar de tener que asociar a cada vista con una variable manualmente.

MainActivity.kt

```
Val binding:ActivityMainBinding
```

```
binding.name.text = ...  
binding.age.text = ...  
binding.loc.text = ...
```

initialize binding



activity_main.xml

```
<layout>  
  <ConstraintLayout ... >  
    <TextView  
      android:id="@+id/name"/>  
    <TextView  
      android:id="@+id/age"/>  
    <TextView  
      android:id="@+id/loc"/>  
  </ConstraintLayout>  
</layout>
```

Modificar el archivo de compilación

Para utilizar *DataBinding* tenemos que modificar el archivo *build.gradle.kts* y añadir el siguiente bloque de código, que le indica al compilador que se generen las clases necesarias que nos permitan trabajar con los layouts definidos en XML.

```
android {  
    ...  
    buildTypes {  
        ...  
    }  
    buildFeatures{  
        dataBinding = true  
    }  
    ...  
}
```

build.gradle.kts (:app)

Con cada cambio en el Gradle, recuerda sincronizar el proyecto (Sync Now)

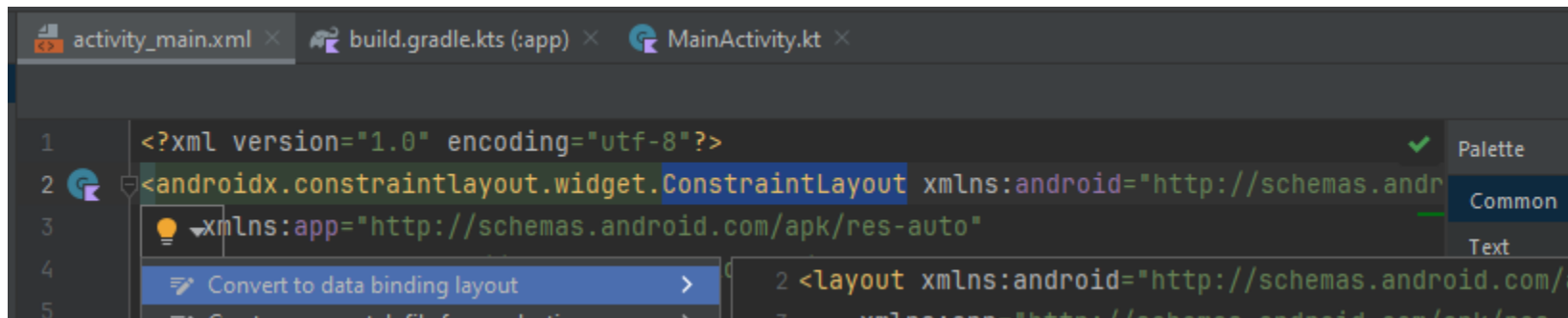
Añadir etiqueta *layout*

Para indicar al compilador qué diseños queremos utilizar con DataBinding, en los XML correspondientes añadimos una etiqueta *layout* que los contenga. La clase generada contendrá referencias a todas las vistas que tengan un ID asignado

```
<layout ... >
  <androidx.constraintlayout.widget.ConstraintLayout>
    <TextView android:id="@+id/usuario" ... />
    <EditText android:id="@+id/contraseña" ... />
  </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

activity_main.xml

Se puede hacer desde las Context Actions del elemento raíz de nuestro diseño. 



Crear la interfaz con DataBinding en la Actividad

En lugar de llamar a `setContentView()` desde la actividad, llamamos a `DataBindingUtil.setContentView()` y le pasamos el Activity (*this*) y el ID del recurso de layout deseado.

```
class MainActivity : AppCompatActivity() {  
    lateinit var binding: ActivityMainBinding  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        // setContentView(R.layout.activity_main)  
        binding = DataBindingUtil.setContentView(this, R.layout.activity_main)  
  
        binding.usuario.text = "Jorge"  
    }  
}
```

MainActivity.kt

Declarar variables directamente en el *layout*

Se pueden crear variables en el propio layout y hacer que las vistas referencien a su valor, simplificando la lógica en la Actividad.

```
<layout>
  <data>
    <variable name="nombre" type="String" />
  </data>
  ...
  <TextView
    android:id="@+id/usuario"
    android:text="@{nombre}" />
  ...
</layout>
```

activity_main.xml

También sirven algunas expresiones simples
como: "@{nombre.toUpperCase()}"

```
override fun onCreate(savedInstanceState: Bundle?) {
  ...
  binding.nombre= "Jorge" // En lugar de: binding.usuario.text = "Jorge"
}
```

MainActivity.kt

Ejercicio 1

Crea una aplicación para llevar la puntuación de un partido de baloncesto utilizando DataBinding.

La aplicación debe tener un diseño como el que se muestra a la derecha, con una columna para cada equipo y un botón para resetear.

En la columna de cada equipo se mostrará el nombre del equipo, su marcador actual y tres botones que incrementarán el marcador cuando se anote un tiro libre, un tiro de campo o un triple.

¿Qué pasa cuando giramos la orientación del dispositivo?

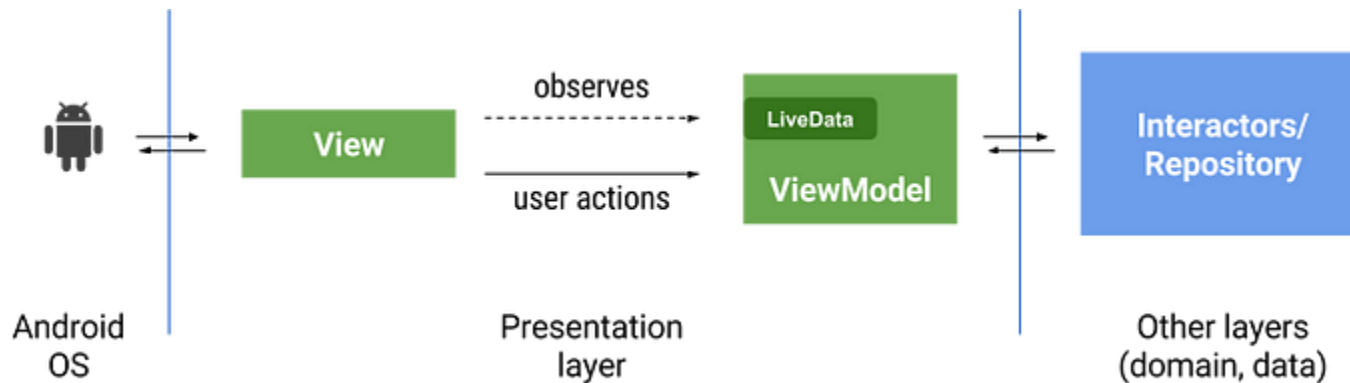


ViewModel

Model - View - View Model (MVVM)

MVVM es un patrón de diseño que tiene por finalidad separar la parte de la interfaz del usuario (View) de la parte de la lógica del negocio (Model), logrando así que la parte visual sea totalmente independiente.

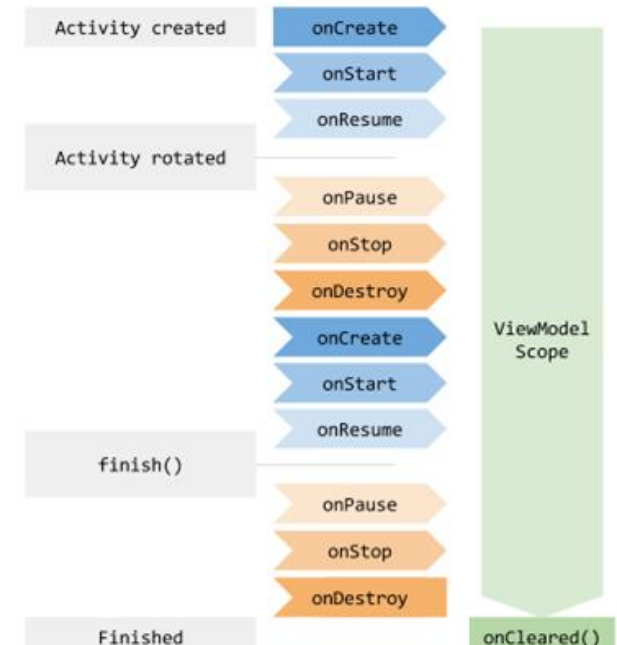
El ViewModel es el tercer componente y se encarga de interactuar como puente entre la Vista y el Modelo.



ViewModel

Un ViewModel permite separar la propiedad de los datos de la lógica del controlador de la interfaz de usuario:

- Prepara los datos para la interfaz de usuario
- No debe hacer referencia a actividades, fragmentos o vistas en la jerarquía de vistas
- Asociado a un ciclo de vida (que tienen la actividad y el fragmento)
- Permite que los datos sobrevivan a los cambios de configuración
- Sobrevive mientras el ámbito está vivo



Añadir dependencias de ViewModel

Para habilitar ViewModel en nuestras aplicaciones, necesitamos añadir algunas dependencias al archivo Gradle de nuestro módulo.

build.gradle.kts (:app)

```
dependencies {  
    val activity_version = "1.8.2"  
    val lifecycle_version = "2.7.0"  
    ...  
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version")  
    implementation("androidx.activity:activity-ktx:$activity_version")  
    ...  
}
```


Crear el ViewModel

Para crear un ViewModel hay que crear una clase que extienda la clase abstracta ViewModel.

```
class LoginViewModel: ViewModel() {  
    var nombre: String = "Jorge"  
    var contrasena: String = ""  
  
    fun cambiarNombre(nuevo: String){  
        nombre = nuevo  
    }  
}
```

LoginViewModel

abstract class ViewModel

Public constructors

<init>()

ViewModel is a class that is responsible for preparing and managing the data for an [Activity](#) or a [Fragment](#).

Protected methods

open [Unit](#)

[onCleared\(\)](#)

Usar el ViewModel en la Actividad

En la Actividad, instanciamos el ViewModel y modificamos la lógica para que las vistas que muestran la información hagan referencia a los atributos de la clase ViewModel.

Cualquier cambio en los datos se hace a través de llamadas a funciones del ViewModel.

```
class MainActivity : AppCompatActivity() {  
    val viewModel: LoginViewModel by viewModels()  
    override fun onCreate(savedInstanceState: Bundle?) {..  
        // binding.usuario.text = "Jorge"  
        binding.nombre = viewModel.nombre  
  
        binding.button.setOnClickListener{  
            //binding.nombre = "Jonas"  
            viewModel.cambiarNombre("Jonas")  
        }  
    }  
}
```

MainActivity.kt

Ejercicio 2

Siguiendo con el ejercicio anterior:

1. Crea un ViewModel que permita:
 1. Almacenar el estado de los marcadores de ambos equipos.
 2. Incrementar la puntuación de cada equipo en 1, 2 o 3 puntos.
 3. Resetear los marcadores.
2. Adapta la MainActivity para que haga uso del ViewModel implementado.

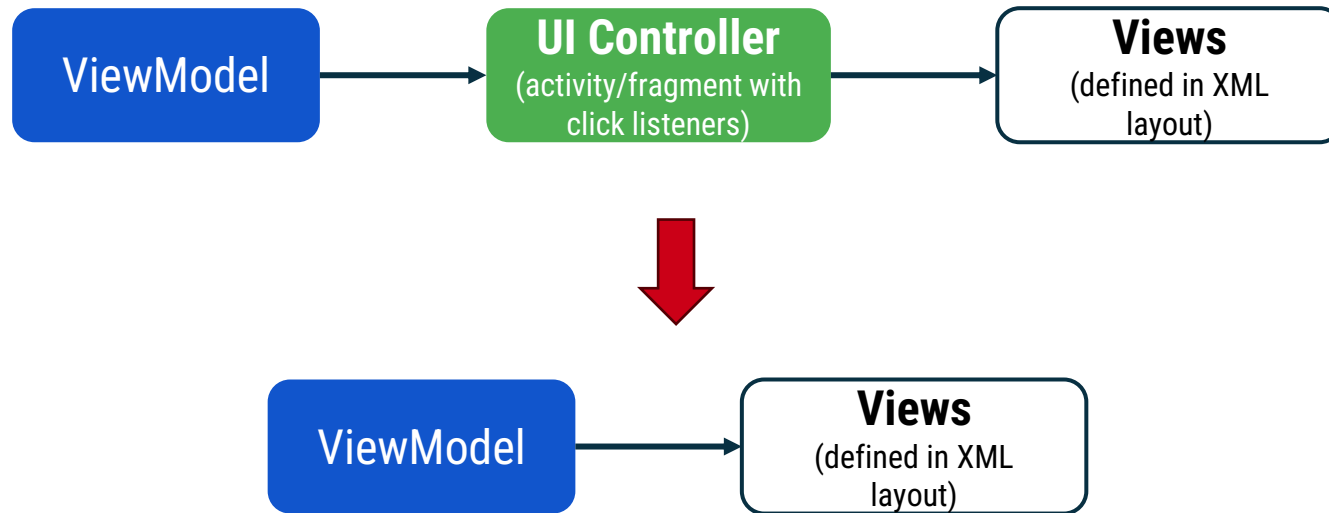
¿Qué pasa cuando pulsamos en los botones?

¿Qué pasa cuando giramos la orientación del dispositivo?

Conectando el XML con el ViewModel

Los objetos ViewModel contienen los datos de la aplicación. En el diseño, sería más sencillo si la instancia ViewModel se comunicara directamente con las vistas, sin depender de los controladores de interfaz de usuario como intermediarios.

Al pasar el ViewModel al DataBinding, se pueden automatizar parte de la comunicación entre las vistas y los objetos ViewModel.



Viewmodel con DataBinding

Para utilizar el ViewModel con DataBinding, se debe declarar una variable en el layout con el tipo correspondiente y asignar el viewModel a dicha variable en la actividad.

```
<layout>
  <data>
    <variable name="viewModel" type="urjc.lsmu.viewmodelsimple.LoginViewModel" />
  </data>
  ...
```

activity_main.xml

```
class MainActivity : AppCompatActivity() {
  lateinit var binding: ActivityMainBinding
  val viewModel: LoginViewModel by viewModels()
  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = DataBindingUtil.setContentView(this, R.layout.activity_main)
    binding.viewModel = viewModel
    ...
  }
}
```

MainActivity.kt

Actualizando la UI con el ViewModel

De esta forma, las vistas podrán hacer referencia a los atributos del ViewModel.

```
<TextView
    android:id="@+id/usuario"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@{viewModel.nombre}" />
```

actviity_main.xml

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = DataBindingUtil.setContentView(this, R.layout.activity_main)
    binding.viewModel = viewModel
    // binding.usuario.text = viewModel.nombre
    binding.button.setOnClickListener{
        viewModel.cambiarNombre("Laura")
        binding.usuario.text = viewModel.nombre
    }
    ...
}
```

MainActivity.kt

Seguimos necesitando
actualizar el Textview
con cada cambio

Ejercicio 3

Siguiendo con el ejercicio anterior, modifica la aplicación para que haga uso del ViewModel a través de DataBinding.

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Patrón Observer (LiveData)



Universidad
Rey Juan Carlos

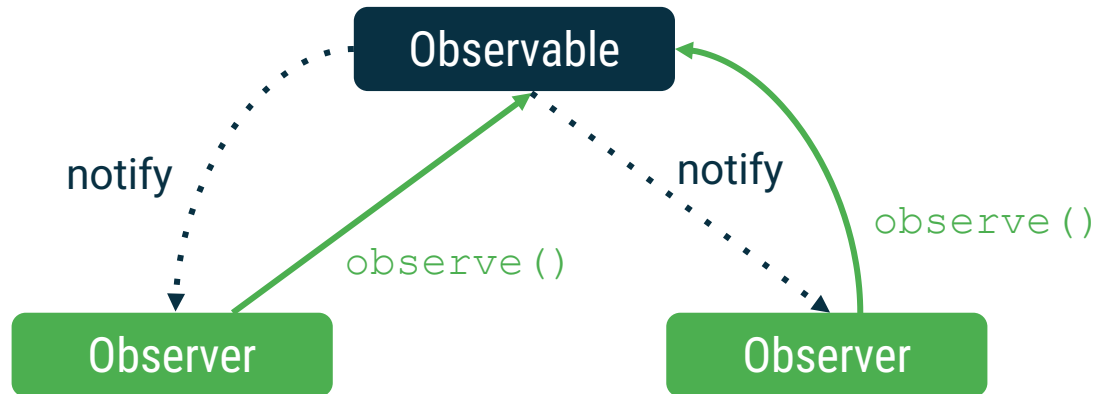
Observer

Patrón de diseño *Observer*

La vinculación de datos (DataBinding) ha reducido algunas de las asignaciones y llamadas a funciones en nuestras interfaces de usuario. Sin embargo, todavía tenemos que actualizar explícitamente las vistas con los nuevos valores del ViewModel cuando cambian. Hay una forma mejor.

El patrón de diseño *Observer* consiste en que un objeto (sujeto u observable) mantiene una lista de objetos dependientes (observadores) para notificar cuando se producen cambios de estado en el sujeto.

Patrón de diseño *Observer* (cont)



- El sujeto (*observable*) mantiene una lista de observadores a los que notificar los cambios de estado.
- Los observadores reciben los cambios de estado del sujeto y ejecutan el código apropiado.
- Los observadores pueden añadirse o eliminarse en cualquier momento.

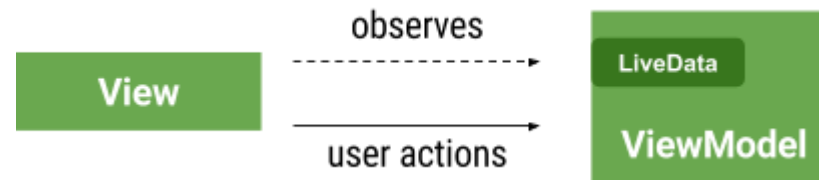
LiveData

LiveData

LiveData es una envoltura alrededor de cualquier tipo de datos (`LiveData<Int>` contiene un *Int*). De acuerdo con el patrón *Observer*, LiveData **es el sujeto observable**.

Los observadores serían controladores de interfaz de usuario, como Activity o Fragment, que esperan cambios en los datos subyacentes para actualizar la interfaz.

- Es un soporte de datos consciente del ciclo de vida que puede ser observado
- A menudo utilizado por ViewModels para contener campos de datos individuales



LiveData

LiveData no tiene métodos públicos para actualizar los datos almacenados. Se puede pensar que es "sólo lectura". Por otro lado, la clase *MutableLiveData* expone los métodos públicos *setValue(T)* y *postValue(T)*.

Habitualmente, *MutableLiveData* es utilizado como una variable privada en el *ViewModel*, que sólo expone un objeto *LiveData* inmutable a los observadores.

Así, el *ViewModel* es el responsable de implementar cualquier lógica y actualizar el *MutableLiveData*, mientras que el papel de la interfaz de usuario es sólo reaccionar a los cambios de valor de *LiveData*.

LiveData<T>	MutableLiveData<T>
● <code>getValue()</code>	● <code>getValue()</code> ● <code>postValue(value: T)</code> ● <code>setValue(value: T)</code>

LiveData en ViewModel

Se actualiza el código del *ViewModel* para utilizar *LiveData<T>* en lugar del tipo de dato *T* original.

Para evitar que ningún objeto externo u observable cambie los datos del *ViewModel*, se declara un *MutableLiveData* privado y un *LiveData* público que no pueden ser reasignados.

```
class LoginViewModel: ViewModel() {  
    private val _nombre = MutableLiveData<String>("Jorge")  
    val nombre: LiveData<String>  
        get() = _nombre  
  
    // La misma estructura pero para contraseña [...]  
  
    fun cambiarNombre(nuevo: String){  
        _nombre.value = nuevo  
    }  
}
```

LoginViewModel.kt

Observando cambios en *LiveData*

En la Actividad, creamos un observador para el dato en cuestión, implementando la lógica a ejecutar cuando se reciba la notificación de un cambio. Luego, se asocia al campo observable del ViewModel.

Las acciones del usuario que modifiquen los datos subyacentes deben lanzar llamadas a métodos del ViewModel (por ejemplo, el click en un botón).

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = DataBindingUtil setContentView(this, R.layout.activity_main)  
    binding.viewModel = viewModel  
    binding.button.setOnClickListener {  
        viewModel.cambiarNombre("Laura")  
    }  
  
    val nombre_observer = Observer<String> { newValue ->  
        binding.usuario.text = newValue.toString()  
    }  
    viewModel.nombre.observe(this, nombre_observer)  
}
```

MainActivity.kt

El primer parámetro de *observe* es el *lifecycleOwner*, que indica a qué ciclo de vida se asocia el observador (una actividad o fragmento)

Ejercicio 1

Modifica la aplicación del marcador de un partido de baloncesto, desarrollada en la sesión anterior, para que al pulsar los botones se modifiquen los puntos del equipo correspondiente:

- Adapta el ViewModel para que los datos estén contenidos en LiveData
- Añade observadores en la actividad principal para reaccionar ante los cambios en los datos del ViewModel



Asociación directa Layout-LiveData

Utilizando *DataBinding* podemos ahorrarnos la declaración explícita de observadores en la actividad.

De esta forma, podemos conectar directamente una vista del layout (.xml) con el valor del sujeto observable (LiveData), consiguiendo que la vista se actualice cuando el dato observado cambie.

Para ello, hay que introducir cambios tanto en el diseño en .xml como en la lógica de nuestra actividad.

Conectar el XML con datos observables

Si asignamos a una vista el valor del atributo LiveData del ViewModel (accesible desde el XML) se elimina la necesidad de declarar un observador en la Actividad.

```
<layout>
  <data>
    <variable name="viewModel" type="urjc.lsmu.viewmodelsimple.LoginViewModel" />
  </data>

  <ConstraintLayout ...>

    <TextView
      android:id="@+id/usuario"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@{viewModel.nombre.toString()}" />

    ...
  </ConstraintLayout>
</layout>
```

MainActivity.kt

Definir el ciclo de vida de los observadores

Después de asignar el objeto *viewModel* al *DataBinding* para hacerlo accesible desde nuestro layout (.xml), hay que especificar el *lifecycleOwner* (la actividad o fragmento) al que va asociado, tal y como se hace al declarar los *observers* de forma explícita.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = DataBindingUtil.setContentViews(this, R.layout.activity_main)  
    binding.viewModel = viewModel  
    binding.lifecycleOwner = this  
  
    binding.button.setOnClickListener {  
        viewModel.cambiarNombre("Laura")  
    }  
}
```

MainActivity.kt

El funcionamiento de esta actividad es equivalente al de la anterior. Ahora, los observadores se generan implícitamente en el *DataBinding*.

Ejercicio 2

Siguiendo con el ejercicio anterior, modifica la aplicación para hacer un enlazado directo entre la vista (xml) y los datos del ViewModel sin necesidad de declarar observadores en la actividad principal.



Ejercicio 3

Crea una aplicación para gestionar una lista de tareas. Sus requisitos son:

1. La app debe tener dos actividades:
 - Una actividad principal que muestra la lista de tareas y tiene un FAB que permite lanzar la actividad secundaria.
 - Una actividad secundaria para añadir tareas nuevas a la lista o editar las ya existentes. Ambas operaciones deben poder ser canceladas.
2. Cada tarea debe contar con un título, una descripción, y un estado (completada/por hacer):
 - En la lista de la actividad principal cada item mostrará únicamente una checkbox indicando si está completada, y el título de la tarea.
 - Si se hace click en la checkbox, se debe cambiar su estado. Si se hace click en el título, se debe lanzar la actividad para editar la tarea.
3. La app debe implementarse utilizando ViewModel, DataBinding y LiveData.

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Persistencia en BBDD (Room)



Universidad
Rey Juan Carlos

Persistencia en Android

Android utiliza un sistema de archivos similar a los sistemas de archivos basados en disco de otras plataformas.

El sistema ofrece varias opciones para que guardes los datos de tu aplicación:

Archivos específicos de la aplicación: almacena archivos destinados únicamente al uso de tu app. Ejemplos: archivos de datos estructurados (archivos JSON), archivos de texto sin formato, archivos multimedia.

Archivos compartidos: almacena archivos que tu app pretende compartir con otras apps, como archivos multimedia o documentos.

Preferencias: almacena datos sencillos en pares clave-valor (SharedPreferences).

Bases de datos: almacena datos estructurados en una base de datos privada para tu aplicación.

Bases de datos

¿Qué es una base de datos?

Una base de datos **es una colección de datos estructurados** a los que se puede acceder, buscar y organizar fácilmente. Los datos **se almacenan en tablas**, y cada tabla contiene campos de información relacionados.

Base de datos de ejemplo

Persona
_id
nombre
edad
dni

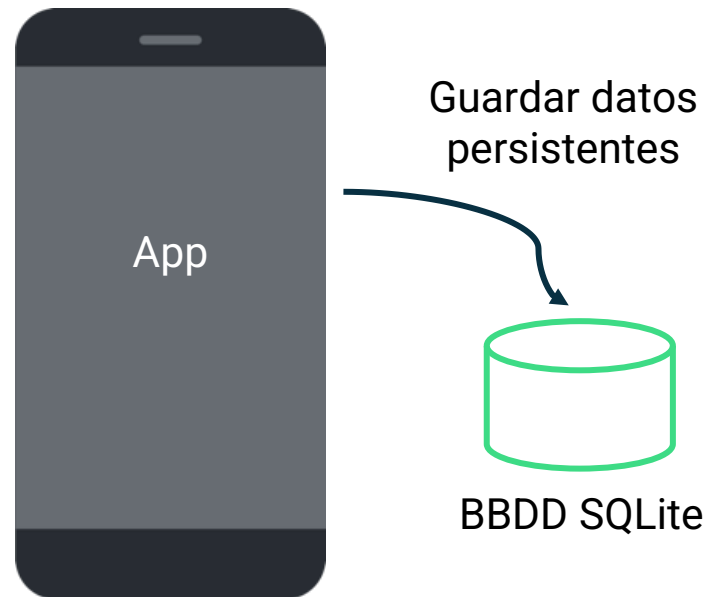
Vivienda
_id
referenciacatastral
superficie
habitaciones

Structured Query Language (SQL)

Para **interactuar con una base de datos relacional**, utilizamos SQL (o Lenguaje de Consulta Estructurado), que es un lenguaje específico para bases de datos.

En Android se utiliza SQLite, que está basado en el estándar SQL pero es más apropiado para dispositivos embebidos. Lo usaremos para:

- Crear nuevas tablas
- Consultas datos
- Insertar datos
- Actualizar datos
- Borrar datos



Ejemplos de comandos SQLite

Leer datos (consultar)

```
SELECT * from colors;
```

Insertar datos

```
INSERT INTO colors VALUES ("red", "#FF0000");
```

Actualizar datos

```
UPDATE colors SET hex="#DD0000" WHERE name="red";
```

Borrar datos

```
DELETE FROM colors WHERE name = "red";
```

En *View > Tool Windows > App Inspection* se puede interactuar con la base de datos de nuestra aplicación, inspeccionarla, hacer queries...

Librería Room

Añadir las dependencias

Para utilizar la librería *Room* hay que añadir las dependencias necesarias en los archivos *build.gradle.kts* del proyecto.

```
plugins {  
    id("org.jetbrains.kotlin.kapt") version "1.9.22" apply false  
    ...  
}
```

build.gradle.kts (Project)

```
plugins {  
    id("org.jetbrains.kotlin.kapt")  
}  
  
dependencies {  
    implementation("androidx.room:room-runtime:2.6.1")  
    kapt("androidx.room:room-compiler:2.6.1")  
    implementation("androidx.room:room-ktx:2.6.1")  
}
```

build.gradle.kts (Module)

¿Qué es Room?

Es una biblioteca de persistencia que brinda **una capa de abstracción para SQLite**, permitiendo acceder a la BBDD de forma simplificada y evitando errores.

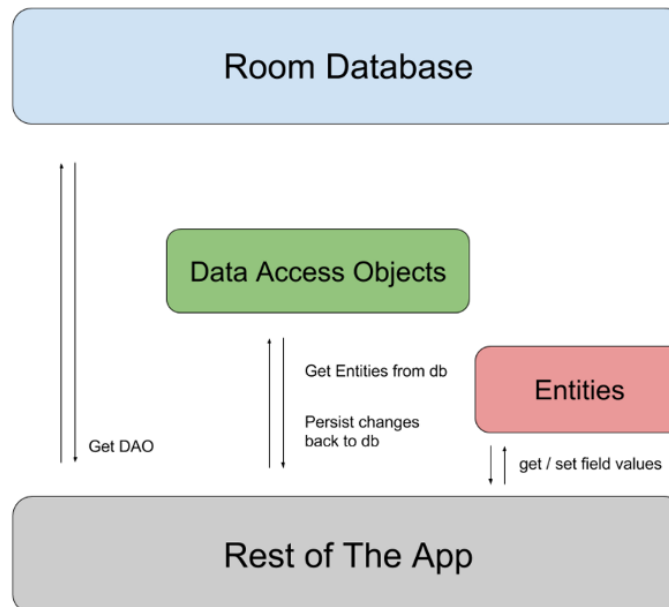
En particular, Room brinda los siguientes beneficios:

- Verificación en tiempo de compilación de las consultas en SQL
- Anotaciones que minimizan el código necesario
- Rutas de migración de bases de datos optimizadas

Componentes de Room

Hay tres componentes principales en Room:

- **Entidad:** representa una tabla dentro de la base de datos
- **DAO:** contiene los métodos utilizados para acceder a la base de datos
- **Base de datos:** contiene el soporte de la BBDD y es el principal punto de acceso a los datos relacionales persistentes de la aplicación



Anotaciones

Las anotaciones son notas que explican o resaltan palabras o pasajes de un texto.

En los lenguajes de programación, las anotaciones **adjuntan metadatos** al código y pueden **proporcionar información adicional al compilador**. En Java, es habitual ver la anotación `@Override` cuando se sobrescriben métodos de una superclase en una subclase.

Room utiliza esta información para **generar código automáticamente**.

Por ejemplo: `@Entity` marca las entidades, `@Database` las bases de datos., y `@Dao` los objetos de acceso a las BBDD

Entity

Las entidades son **clases que se mapean a tablas SQLite**. Se implementan generalmente como clases de datos (***data class***).

La clase se anota como `@Entity`, indicando el nombre de la tabla. Cada atributo de la clase representa una columna de la tabla. El anotado como `@PrimaryKey` será único y puede autogenerarse. El resto se anotan como `@ColumnInfo` si queremos que el nombre del atributo y el de la columna en la tabla difieran.

```
@Entity(tableName = "pelicula_table")
data class Pelicula (
    @PrimaryKey(autoGenerate = true)
    val peliculaId: Int?,
    @ColumnInfo(name = "pelicula_titulo")
    val titulo: String,
    @ColumnInfo(name = "pelicula_anyo")
    val anyo: Int,
    val director: String
)
```

Pelicula.kt

pelicula_table
peliculaId
pelicula_titulo
pelicula_anyo
director

Data Access Object (DAO)

Para **acceder a los datos** de tu aplicación usando la librería de persistencia Room, trabajas con objetos de acceso a datos, o DAOs.

Una interfaz DAO define las interacciones deseadas con la base de datos, en lugar de utilizar constructores de consultas o consultas directas.

Trabajar con clases DAO en lugar de acceder directamente a la base de datos presenta importantes ventajas:

- Room crea la implementación del DAO en tiempo de compilación.
- Room verifica todas sus consultas DAO en tiempo de compilación.

Implementar un DAO

Anotamos la interfaz con **@Dao** para que el compilador genere la implementación automáticamente

```
@Dao
interface PeliculaDAO {
    @Insert
    fun insert(vararg pelicula: Pelicula)
```

vararg permite insertar más de un dato en una sola transacción

PeliculaDAO.kt

```
    @Update
    fun update(pelicula: Pelicula)
```

```
    @Delete
    fun delete(pelicula: Pelicula)
```

```
    @Query("SELECT * FROM pelicula_table")
    fun getAll(): LiveData<List<Pelicula>>
```

Anotamos cada método con *@Insert*, *@Update*, *@Delete* o *@Query* según corresponda

Para las consultas, los parámetros se inyectan con el formato :param

```
    @Query("SELECT * FROM pelicula_table WHERE pelicula_titulo = :nombre")
    fun getByName(nombre: String): LiveData<List<Pelicula>>
```

```
}
```

Crear una base de datos Room

Declaramos una clase abstracta que herede de *RoomDatabase*

Anotamos la clase con **@Database** e indicamos sus entidades

PeliculasRoomDatabase.kt

```
@Database(entities = [Pelicula::class], version = 1)
abstract class PeliculasRoomDatabase : RoomDatabase(){
    abstract fun peliculaDAO(): PeliculaDAO

    companion object{
        @Volatile
        private var INSTANCE: PeliculasRoomDatabase? = null

        fun getInstance(context: Context): PeliculasRoomDatabase{
            ...
        }
    }
}
```

Declaramos un método abstracto sin argumentos que devuelve el DAO

Declaramos un *singleton* que nos dé una instancia de la BBDD

@Volatile hace que la variable nunca se almacene en caché (siempre en RAM) por lo que los cambios siempre son visibles para todos los hilos

Crear la instancia de la base de datos

Usamos `Room.databaseBuilder()` para crear una base de datos usando el contexto de la aplicación, la clase de la BBDD y el nombre de la base de datos.

```
fun getInstance(context: Context): PeliculasRoomDatabase{
    val tempInstance = INSTANCE
    if(tempInstance != null){
        return tempInstance
    }
    synchronized(this){
        val instance = Room.databaseBuilder(
            context.applicationContext,
            PeliculasRoomDatabase::class.java, "pelicula_database"
        ).allowMainThreadQueries().build()
        INSTANCE = instance
        return instance
    }
}
```

PeliculasRoomDatabase.kt

`@synchronized` garantiza que solo un hilo entra en este bloque a la vez, asegurando que únicamente exista una instancia de la BBDD

`allowMainThreadQueries` NO SE UTILIZA. Solo para pruebas

Obtener y usar el DAO

Para conectarnos con la base de datos y hacer peticiones debemos:

1. Crear una instancia del objeto DAO correspondiente.
2. Utilizar alguno de sus métodos públicos para consultar, insertar, actualizar o borrar registros.

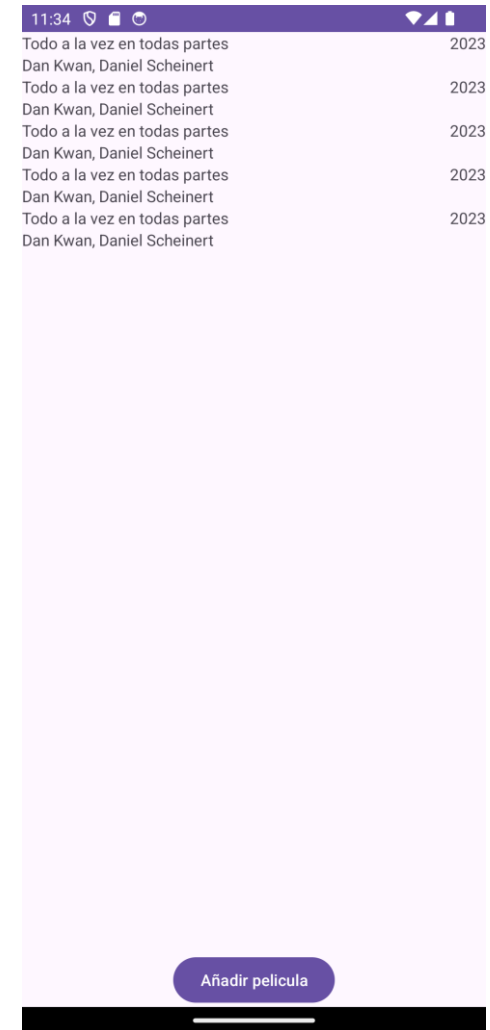
```
val películasDAO = PelículasRoomDatabase.getInstance(application).películaDAO()  
películas = películasDAO.getAll()
```

Ejercicio 1

Utilizando las implementaciones de Entidad, DAO y RoomDatabase de las diapositivas anteriores:

- Crea una aplicación con una única actividad que muestre una lista de películas en un RecyclerView tal y como se muestra en la captura de la derecha.
- El botón inferior debe añadir una película nueva (una predefinida, siempre la misma)
- Al cerrar la aplicación por completo y volverla a abrir, se deben mostrar las películas anteriormente añadidas.

Importante: para simplificar el ejercicio, la lista de películas observable se declarará en la *MainActivity*, prescindiendo así del *ViewModel*.



Crear ViewModel con acceso a la BBDD Room

En lugar de tener la BBDD en la Actividad, se modifica el VM para que tenga acceso a la BBDD Room a través de su DAO. Para ello, el VM necesita un *Context*.

```
class PeliculasViewModel(context: Context): ViewModel() {  
    private val myDAO: PeliculaDAO // DAO para interactuar con la BBDD  
    val peliculas: LiveData<List<Pelicula>> // Lista observable pública expuesta a la Actividad  
    init {  
        myDAO = PeliculasRoomDatabase.getInstance(context).peliculaDAO() // Inicializar DAO  
        peliculas = myDAO.getAll() // Asociamos la lista con la tabla de Peliculas de la BBDD  
    }  
  
    fun addPelicula(pelicula: Pelicula){  
        myDAO.insert(pelicula)  
    }  
  
    fun delPelicula(pelicula: Pelicula){  
        myDAO.delete(pelicula)  
    }  
}
```

PeliculasViewModel.kt

ViewModelFactory para VMs con parámetros

Para poder instanciar un ViewModel que recibe parámetros en su constructor, hay que crear una clase que herede de *Factory*, y devolver el VM en *create()*:

```
// Clase Factory para instanciar el ViewModel
// Recibe el mismo parámetro que el ViewModel y se lo pasa al crearlo
class PeliculasViewModelFactory(private val context: Context) : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        return PeliculasViewModel(context) as T
    }
}
```

PeliculasViewModelFactory.kt

Para utilizar el VM se llama a *ViewModelProvider*, que nos permite construirlo a través de la factoría creada anteriormente, pasándole los parámetros adecuados:

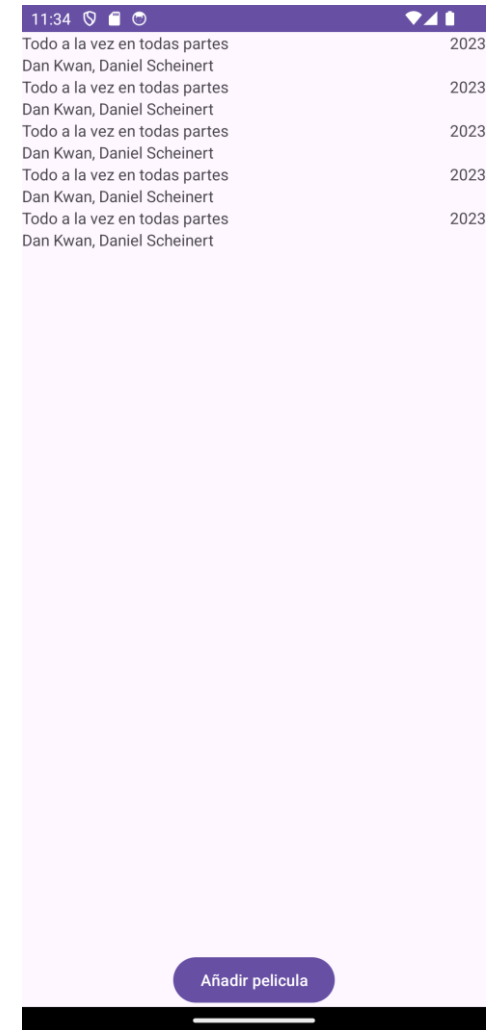
```
var viewModel = ViewModelProvider(
    this,
    PeliculasViewModelFactory(applicationContext)
)[PeliculasViewModel::class.java]
```

Ejercicio 2

Adapta la solución implementada en el ejercicio anterior para incluir un ViewModel que haga de intermediario entre la interfaz (Activity) y los datos (BBDD Room).

Para ello:

- Crea el ViewModel con conexión a una BBDD
- Crea la Factory asociada
- Adapta la actividad para que el RecyclerView se actualice cuando cambie la lista de películas del ViewModel



Ejercicio 3

Desarrolla una aplicación en Android para gestionar una biblioteca personal de libros, utilizando una base de datos local con Room.

La aplicación debe mostrar los libros en cuadrícula, utilizando un RecyclerView. Los libros tienen que estar almacenados en una base de datos que se accederá de forma segura a través de un ViewModel.

Para ello:

- La actividad principal mostrará los libros guardados en Room. Cada libro se verá con una carátula (siempre la misma imagen) y su título debajo
- El evento *onClick* en un libro nos llevará a una actividad donde se muestren todos los detalles del mismo
- El evento *onLongClick* borrará el libro de la base de datos
- Una actividad secundaria permitirá añadir libros nuevos a la base de datos

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Patrón repositorio



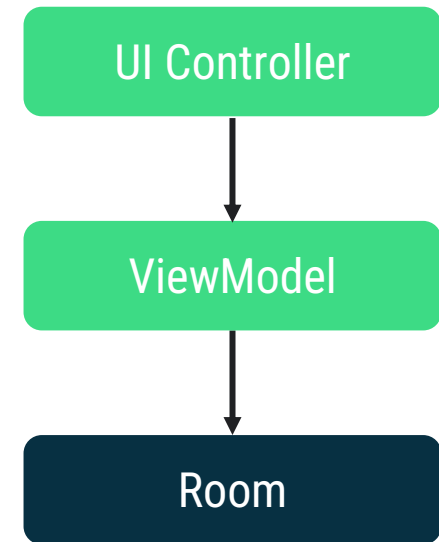
Universidad
Rey Juan Carlos

App con múltiples fuentes de datos

En las aplicaciones que hemos construido hasta ahora, nuestros objetos del modelo de vista contenían referencias directas a objetos DAO para consultar la base de datos local.

Para la mayoría de aplicaciones, donde habitualmente se necesita obtener datos de recursos externos (por ejemplo, la red), se hace muy difícil escalar la aplicación con la arquitectura existente.

La lógica que tendríamos que añadir al modelo de vista sería bastante compleja, y **no queremos que el ViewModel sea el responsable de gestionar la recuperación de datos.**



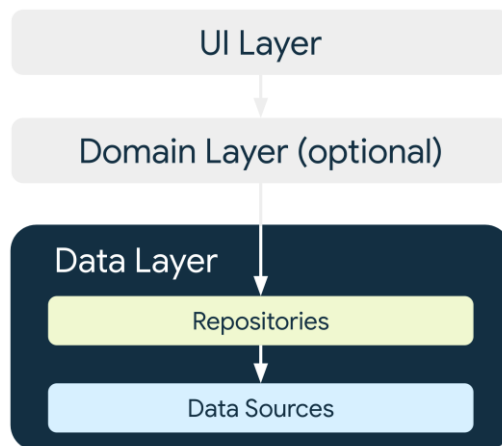
Patrón repositorio

Patrón repositorio

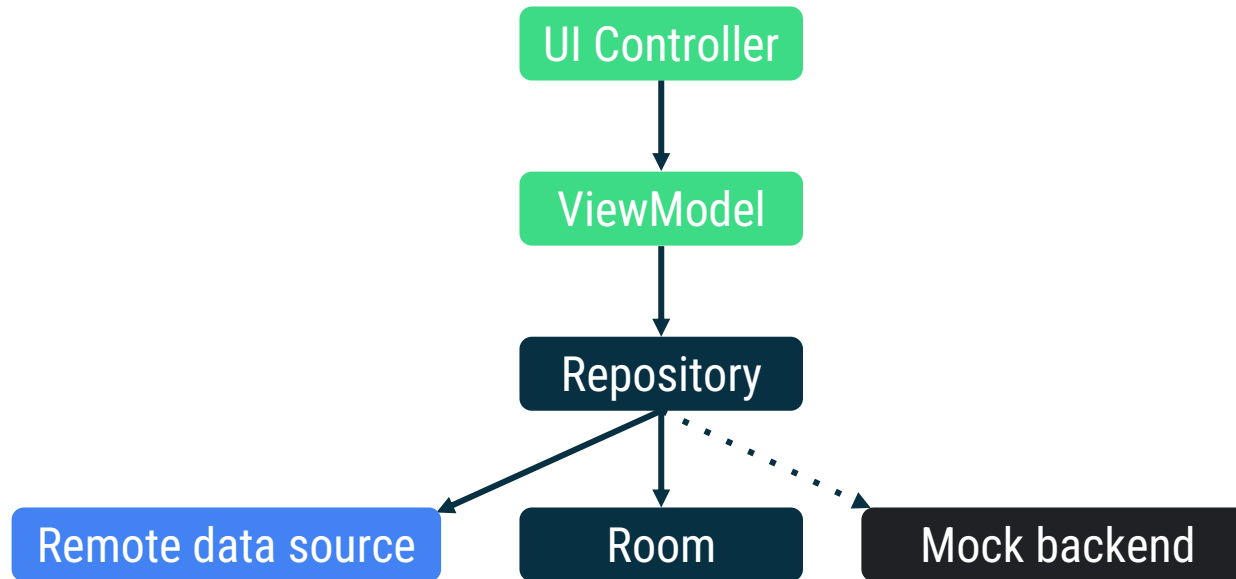
El patrón de repositorio en Android es un enfoque de diseño que proporciona una forma estructurada y organizada de **gestionar el acceso y la manipulación de datos**.

Sirve como **intermediario entre diferentes fuentes de datos** (como bases de datos, servicios de red) **y el resto de la aplicación**.

El objetivo principal es **abstraer los detalles de acceso a los datos**, promoviendo una arquitectura limpia, la reutilización del código y la facilidad de mantenimiento.



Arquitectura de la app usando Repositorio



- Puede abstraer múltiples fuentes de datos, separándolo de la capa de UI
- Permite una recuperación rápida utilizando la base de datos local mientras se envía una solicitud de red para actualizar los datos (más lento)
- Permite probar las fuentes de datos de manera independiente

Implementación de un repositorio

Cuando se trata de implementar el patrón de repositorio en tu aplicación, **no hay requisitos explícitos** (en términos de una clase que debas extender o métodos que debas declarar). Pero...

debe proporcionar una interfaz común para acceder a los datos:

- Exponer funciones para consultar y modificar los datos subyacentes

dependiendo de tus fuentes de datos, el repositorio puede:

- Mantener una referencia a la DAO, si sus datos están en una base de datos
- Realizar peticiones de red, si se conecta a un servicio web

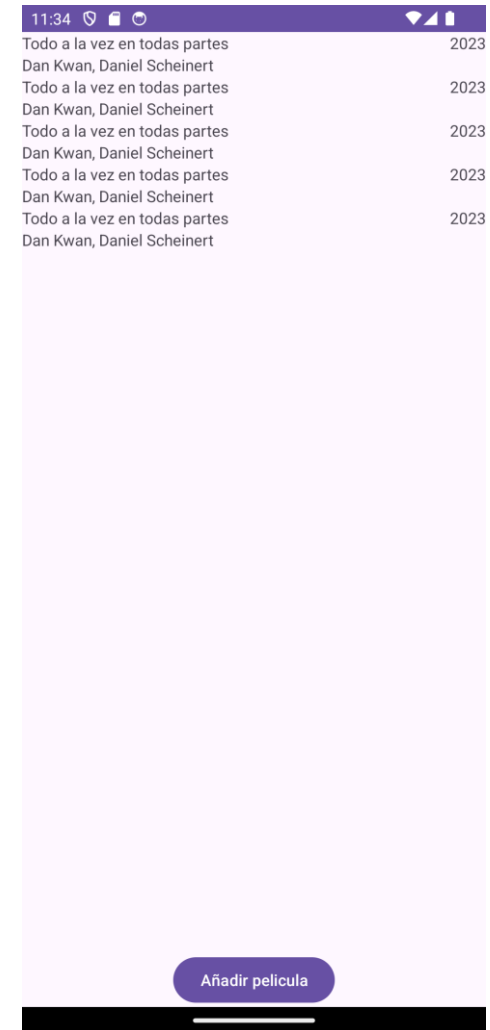
Repositorio simple con Room y API Rest

```
class LibrosRepository(LibrosRepository.kt  
    private val dao: LibrosDAO, // DAO para interactuar con bdd local (Room)  
    private val api: LibrosAPI // API para hacer peticiones a nuestro servidor remoto (Retrofit)  
) {  
    var libros: LiveData<List<Libro>> = dao.getAll()  
    fun getAllLibros(): LiveData<List<Libro>> {  
        if (libros.value?.isEmpty() == true) {  
            var remoteLibros = getLibrosRemote()  
            // TODO Insertar películas remotas en bdd local  
        }  
        return libros  
    }  
  
    // Función para recuperar los datos de películas de un servidor remoto con la API Rest  
    fun getLibrosRemote(): List<Libro> = TODO()  
  
    fun addLibro(libro: Libro) {  
        dao.insert(libro)  
    }  
}
```

Ejercicio 1

Siguiendo con la aplicación de la sesión anterior, donde teníamos una actividad que mostraba las películas almacenadas en una base de datos Room accesible desde un ViewModel:

- Implementa una clase repositorio para que se encargue de las tareas de recuperación de datos de la base de datos Room
- Adapta las clases necesarias en la aplicación para que haga uso del patrón repositorio



Separación modelo de datos-dominio

Además de abstraer las diferentes fuentes de datos de la aplicación y separarlas de la capa de UI (ViewModel), **el repositorio debe emplearse para separar el modelo de datos** (lo almacenado en Room o en remoto) **del modelo de dominio** (datos que usa nuestra aplicación).

El repositorio es responsable de:

- Exponer únicamente los datos de dominio (los utilizados por la app, independientemente de la información almacenada en las diferentes fuentes de datos).
- Interactuar con las fuentes de datos utilizando los objetos adecuados que representen la información almacenada (p. ej. Entity).

Clase dominio vs de fuente de datos

Las entidades definen las columnas de la tabla en la base de datos Room, por lo que sus atributos solo pueden tomar tipos básicos (int, float, string...)

```
@Entity(tableName = "libros")
data class LibroEntity(// Antes llamada Libro !!!
    @PrimaryKey(autoGenerate = true) val id: Int? = null,
    val titulo: String,
    val fecha : String // ejemplo: 2018-03-27
)
```

LibroEntity.kt

Para la lógica interna de nuestra aplicación, eso nos puede suponer una limitación. Por ello, es conveniente diferenciar las clases de dominio de las clases de entidad (Room) o API (Retrofit).

```
data class Libro(
    val id: Int?,
    val titulo: String,
    val fecha: LocalDate
)
```

Libro.kt

Mapecto entre clases

Al separar las clases de datos de dominio de las de fuentes de datos, es necesario implementar funciones para las conversiones entre las mismas.

```
fun LibroEntity.toDomain(): Libro {  
    return Libro(  
        id = this.id,  
        titulo = this.titulo,  
        fecha = LocalDate.parse(this.fecha)  
    )  
}
```

LibroMapper.kt

```
fun Libro.toEntity(): LibroEntity {  
    return LibroEntity(  
        id = this.id,  
        titulo = this.titulo,  
        fecha = this.fecha.toString()  
    )  
}
```

Repositorio como mediador

El repositorio, por tanto, se encarga de **recuperar los datos de la fuente** que sea y **mostrarlos con el formato de datos de dominio** esperado por la aplicación, realizando las transformaciones necesarias.

Para ello, podemos utilizar la función *map*, que permite realizar cambios dentro de un *LiveData* o de una *List*, juntos con las funciones de mapeo implementadas.



```
class LibrosRepository(private val dao: LibroDAO) {  
    fun getAllLibros(): LiveData<List<Libro>> = dao.getAll().map { listaLibros ->  
        listaLibros.map { it.toDomain() } }  
  
    // Resto de funciones expuestas (p. ej.: addLibro, deleteLibro, etc)  
}
```

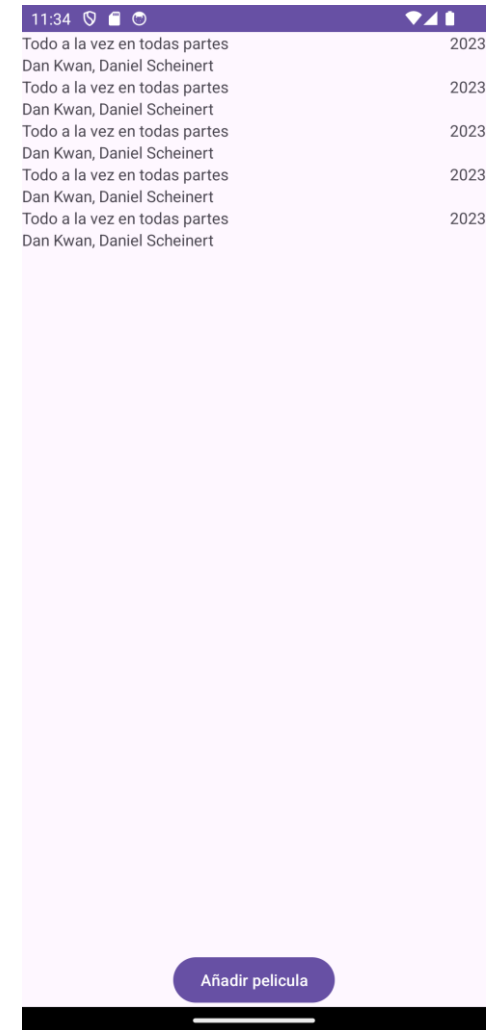
LibrosRepository.kt

Ejercicio 2

Siguiendo con el ejercicio anterior:

- Modifica la BBDD para que el campo *año* sea la fecha de *estreno*, dejando de ser un entero y almacenándose como una String “YYYY-MM-DD”
- Implementa una clase de dominio para la Pelicula, donde el atributo *estreno* sea del tipo [LocalDate](#)
- Implementa las funciones de conversión necesarias entre la entidad y la clase de dominio

Importante: en el RecyclerView la visualización no cambia. Es decir, se seguirá mostrando únicamente el año de estreno de la película, no la fecha completa.



Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Programación asíncrona



Universidad
Rey Juan Carlos

Programación asíncrona en Android

Las aplicaciones usan un subproceso principal para controlar las interacciones con la UI. **Lanzar tareas de larga duración desde este subproceso puede generar bloqueos y falta de respuesta.**

La programación asíncrona nos permite lidiar con múltiples requisitos de las apps móviles:

- Tiempo limitado para realizar tareas y **mantener la capacidad de respuesta**
- Equilibrio con la necesidad de **ejecutar tareas de larga duración**
- **Control sobre cómo y dónde** se ejecutan las tareas

Ejemplos de tareas de larga duración: obtener o sincronizar datos con un servidor remoto, leer o escribir en una base de datos o un archivo, un cálculo pesado...

Coroutines

Coroutines o corrutinas

Una *coroutine* es un **patrón de diseño de concurrencia** disponible en Android para simplificar el código que se ejecuta de forma asíncrona. Las coroutines se añadieron a Kotlin en la versión 1.3.

Las coroutines permiten:

- Mantener la capacidad de respuesta de tu aplicación mientras se gestionan tareas de larga duración
- Simplificar el código asíncrono
- Escribir código de forma secuencial
- Manejar las excepciones con el bloque try/catch

Beneficios de las corrutinas

- **Ligereza:** se pueden ejecutar muchas corrutinas en un solo subproceso debido a la compatibilidad con la suspensión, que no bloquea el subproceso en el que se ejecuta la corrutina. La suspensión ahorra más memoria que el bloqueo y admite muchas operaciones simultáneas.
- **Menos fugas de memoria:** usa la simultaneidad estructurada para ejecutar operaciones dentro de un alcance.
- **Compatibilidad con cancelación integrada:** la cancelación se propaga automáticamente a través de la jerarquía de corrutinas en ejecución.
- **Integración con Jetpack:** muchas bibliotecas Jetpack incluyen extensiones que proporcionan compatibilidad total con corrutinas.

Dependencias de *coroutines*

Para utilizar corrutinas hay que añadir las dependencias necesarias en el archivo *build.gradle.kts (:app)*

```
dependencies {  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.3.9")  
    ...  
}
```

build.gradle.kts (:app)

Funciones para corrutinas (suspend)

Utilice el modificador ***suspend*** para declarar una función como disponible para las **coroutines**. La función sólo se puede invocar **desde dentro de una coroutine o desde otra función de suspensión**.

```
class LibrosRepository(private val dao: LibrosDAO) {  
    ...  
    suspend fun addLibro(libro: Libro){  
        dao.insert(libro)  
    }  
}
```

LibrosRepository.kt

Cuando una corrutina llama a una función marcada con el modificador *suspend*, en lugar de bloquearse hasta que esa función retorna como una llamada normal a una función, **suspende la ejecución hasta que el resultado está listo y entonces se reanuda donde se quedó con el resultado**.

Mientras está suspendida esperando un resultado, **desbloquea el hilo** en el que se está ejecutando **para que otras funciones o coroutines puedan ejecutarse**.

DAO con suspend

Para que las **operaciones con la base de datos** no se hagan en el hilo principal, las funciones del DAO **deben marcarse con *suspend***. Todas las funciones, **excepto las queries que devuelvan un resultado observable** (LiveData), que pueden devolver varios resultados y ya se ejecutan en segundo plano por defecto.

```
@Dao
interface LibrosDao {
    @Query("SELECT * FROM libros")
    fun getAll(): LiveData<List<Libro>>

    @Insert
    suspend fun insert(vararg libro: Libro)

    @Update
    suspend fun update(libro: Libro)

    @Delete
    suspend fun delete(libro: Libro)
}
```

LibrosDAO.kt

Lo mismo sucede con las funciones de la clase Repositorio, encargadas de interactuar con el DAO o una API remota

Dispatchers

Usar *suspend* no le dice a Kotlin que ejecute una función en un hilo de fondo. Las coroutines pueden ejecutarse en el hilo principal.

Kotlin tiene varios **dispatchers (hilos)** que puedes usar para especificar **dónde deben ejecutarse las coroutines**, dependiendo de la tarea que estés realizando:

Dispatcher	Descripción del trabajo	Ejemplo de tarea
Dispatchers.Main	UI y tareas cortas (no bloqueantes)	Actualizar LiveData, llamar a funciones de suspensión
Dispatchers.IO	Tareas de red y disco	BBDS, leer/escribir archivos
Dispatchers.Default	Uso de CPU intensivo	Parsear un JSON

Alcance o *scope*

Las corrutinas **deben ejecutarse en un CoroutineScope**:

- Mantiene un registro de todas las corrutinas iniciadas en él (incluso las suspendidas)
- Proporciona una forma de cancelar las corrutinas en el scope
- Proporciona un puente entre funciones regulares y corrutinas

Por defecto, las corrutinas se lanzan en el GlobalScope, aunque a menudo **es conveniente asociarlas al ciclo de vida del componente que la ha lanzado**. Para ello, las librerías Jetpack tienen algunos alcances definidos, por ejemplo:

- *viewModelScope* para los ViewModel
- *lifecycleScope* para las Actividades o Fragmentos

Lanzar una corrutina

Las corrutinas **se lanzan**, principalmente, **con la función *launch*** de un CoroutineScope. Lanzar **launch no es bloqueante**, por lo que sigue ejecutándose las siguientes instrucciones.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        lifecycleScope.launch {  
            coroutineFunction()  
        }  
        lifecycleScope.launch(Dispatchers.IO) {  
            coroutineFunction2()  
        }  
    }  
}
```

MainActivity.kt

```
suspend fun coroutineFunction(){  
    delay(1000L)  
    Log.d("Main", "coroutine")  
}  
  
suspend fun coroutineFunction2(){  
    CoroutineScope(Dispatchers.IO).launch {  
        Log.d("Main", "coroutine2")  
    }  
}
```


Forzar un Dispatcher en una corrutina

Si queremos que una tarea se haga en un *Dispatcher* concreto independientemente desde donde se lance (por ejemplo, escribir en una bbdd), podemos utilizar la función *withContext*. Al contrario de *launch*, esta función es una función de suspensión.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        lifecycleScope.launch {  
            coroutineFunction()  
        }  
  
        lifecycleScope.launch {  
            coroutineFunction2()  
        }  
    }  
}
```

MainActivity.kt

```
suspend fun coroutineFunction2(){  
    // En el Dispatcher donde la lancen  
    withContext(Dispatchers.IO){  
        // Cambia al Dispatcher IO  
        Log.d("Main", "coroutine2")  
    }  
    // Vuelve al dispatcher original  
}
```

Corrutinas asociadas a un ViewModel

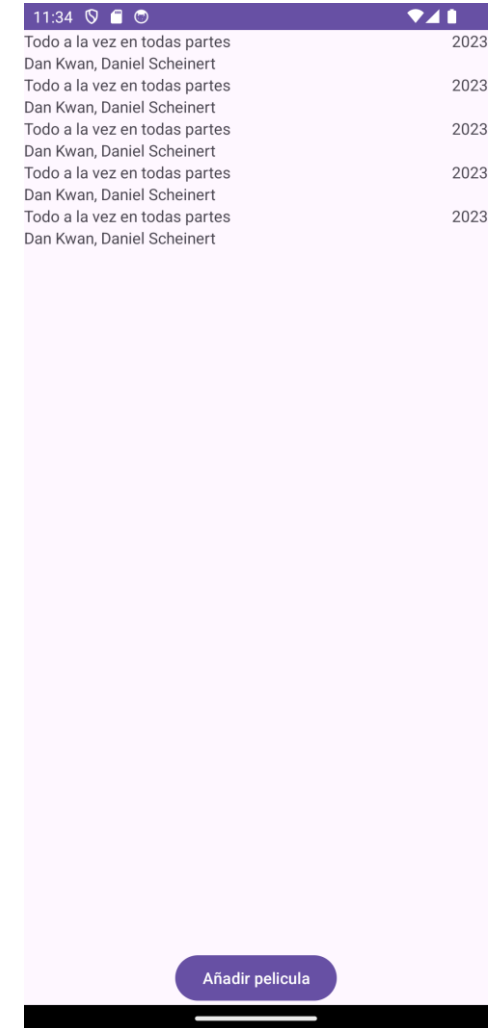
Se define un **CoroutineScope** para cada **ViewModel** de tu aplicación. Puedes acceder a él a través de la propiedad ***viewModelScope*** del **ViewModel**.

Cualquier coroutine lanzada en este ámbito **se cancela automáticamente si se borra el ViewModel**. Las coroutines son útiles aquí para cuando tienes trabajo que necesita hacerse sólo si el **ViewModel** está activo (p.ej., leer datos de Room).

```
class MyViewModel: ViewModel() {  
  
    init {  
        viewModelScope.launch {  
            // Coroutine que se cancela cuando se borra el ViewModel  
        }  
    }  
    ....  
}
```

Ejercicio 1

Siguiendo con la aplicación de la lista de películas desarrollada en las sesiones anteriores, haz los cambios necesarios para que las operaciones sobre la base de datos se realicen en segundo plano.



Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Localización



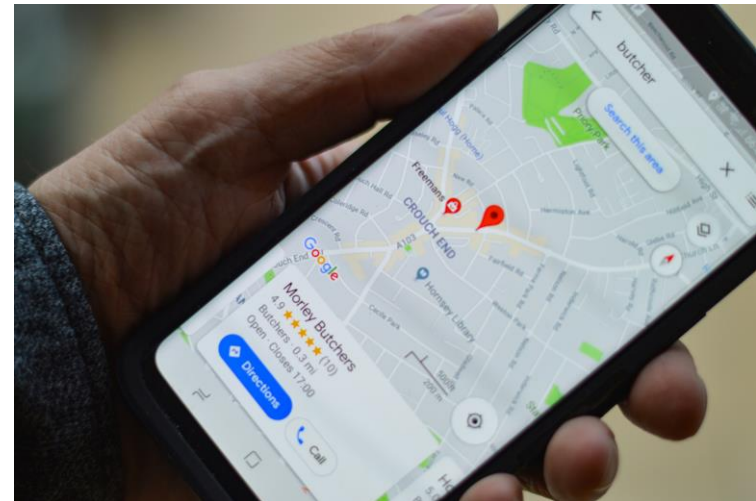
Universidad
Rey Juan Carlos

Localización

Una de las características únicas de las aplicaciones móviles es el conocimiento de la ubicación. Los usuarios de móviles llevan sus dispositivos a todas partes, y añadir el conocimiento de la ubicación a su aplicación ofrece a los usuarios una experiencia más contextual.

Posibles usos de la ubicación:

- Indicaciones en un mapa
- Avisos de peligro
- Restricciones de acceso a contenidos
- Recopilar datos para vendérselos a terceros...



Posición GPS

Tipos de localización

Si tu aplicación contiene una función que comparte o recibe información de ubicación sólo una vez, o durante un periodo de tiempo definido, entonces requiere acceso a la **ubicación en primer plano**. Ejemplos:

- En una aplicación de navegación, para obtener indicaciones giro a giro.
- En una aplicación de mensajería, para compartir la ubicación con otro usuario.

Permisos: ACCESS_COARSE_LOCATION o ACCESS_FINE_LOCATION

Una aplicación necesita acceso a la **ubicación en segundo plano** si una función de la aplicación comparte constantemente la ubicación con otros usuarios o utiliza la *Geofencing API*.

Permiso: ACCESS_BACKGROUND_LOCATION

Dependencias y permisos

Para acceder a la ubicación desde nuestra aplicación debemos configurar las dependencias y los permisos

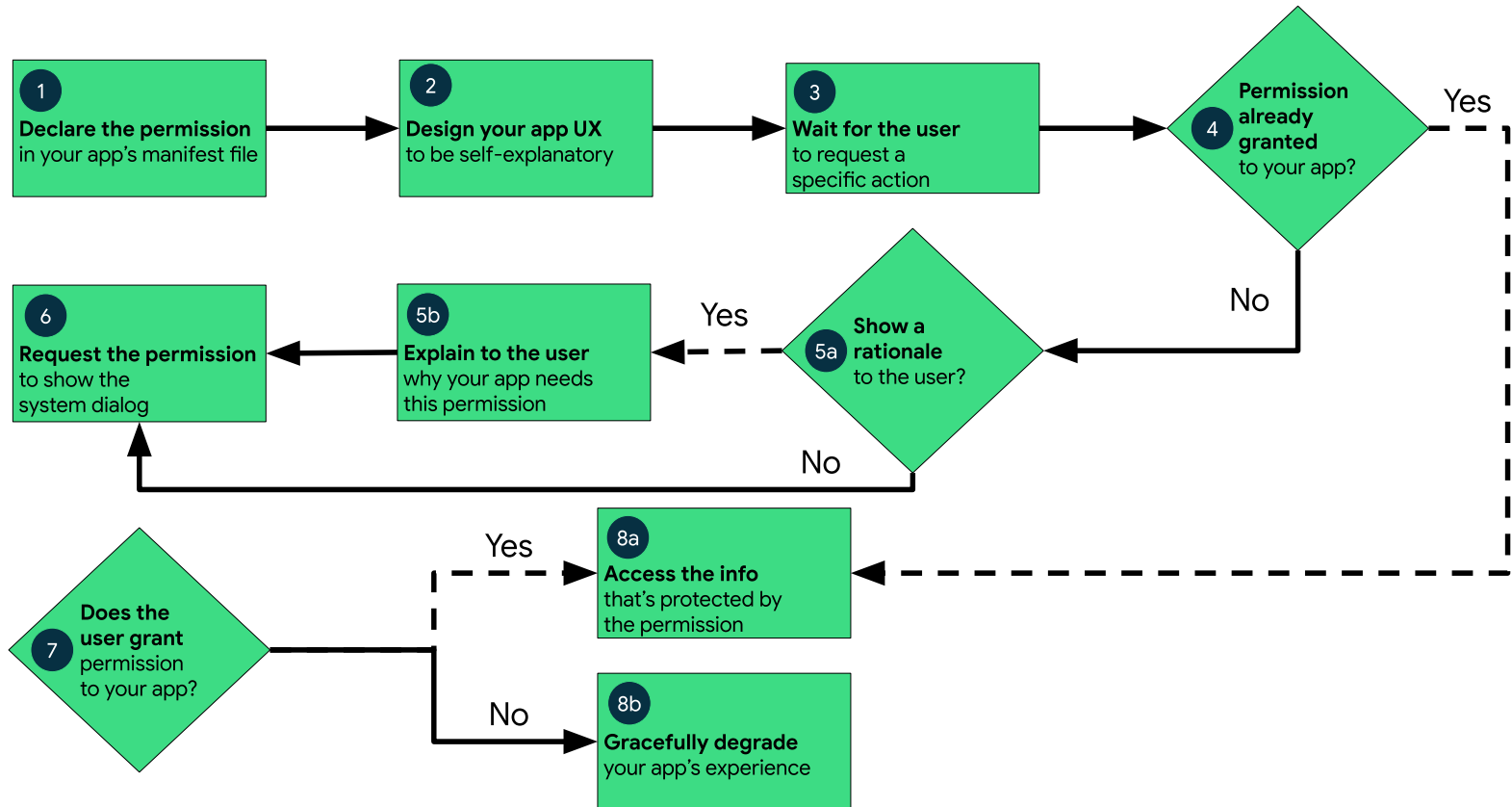
```
dependencies {  
    implementation(libs.play.services.location)  
}
```

build.gradle.kts (:app)

```
<?xml version="1.0" encoding="utf-8"?>  
<manifest xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:tools="http://schemas.android.com/tools">  
  
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />  
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />  
  
    <application  
        ...
```

AndroidManifest.xml

Solicitud de permisos



Comprobar permisos disponibles

Al iniciar la aplicación, **se deben comprobar si los permisos necesarios están disponibles**. Si se ha usado previamente la app, la comprobación será un éxito. Tras la instalación, o si estos han sido revocados, no estarán habilitados.

```
if (ActivityCompat.checkSelfPermission(  
    this,  
    Manifest.permission.ACCESS_FINE_LOCATION  
) == PackageManager.PERMISSION_GRANTED && ActivityCompat.checkSelfPermission(  
    this,  
    Manifest.permission.ACCESS_COARSE_LOCATION  
) == PackageManager.PERMISSION_GRANTED){  
    Log.d("Permisos", "La app tiene los permisos necesarios")  
}else{  
    Log.d("Permisos", "La app no tiene los permisos necesarios")  
}
```

MainActivity.kt

Solicitar permisos al usuario

Si los permisos no están concedidos, hay que solicitarlos y gestionar tanto su aceptación como la negativa del usuario

```
// Configurar la petición de permisos
locationPermissionRequest = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { permissions ->
    when {
        permissions.getDefault(Manifest.permission.ACCESS_FINE_LOCATION, false) -> {
            Log.d("Permisos", "Permisos aceptados")
        } else -> { // Solo localización aproximada o ningún permiso concedido
            Log.d("Permisos", "Permisos rechazados")
        }
    }
}

// Pedir los permisos
locationPermissionRequest.launch(arrayOf(
    Manifest.permission.ACCESS_FINE_LOCATION,
    Manifest.permission.ACCESS_COARSE_LOCATION))
```

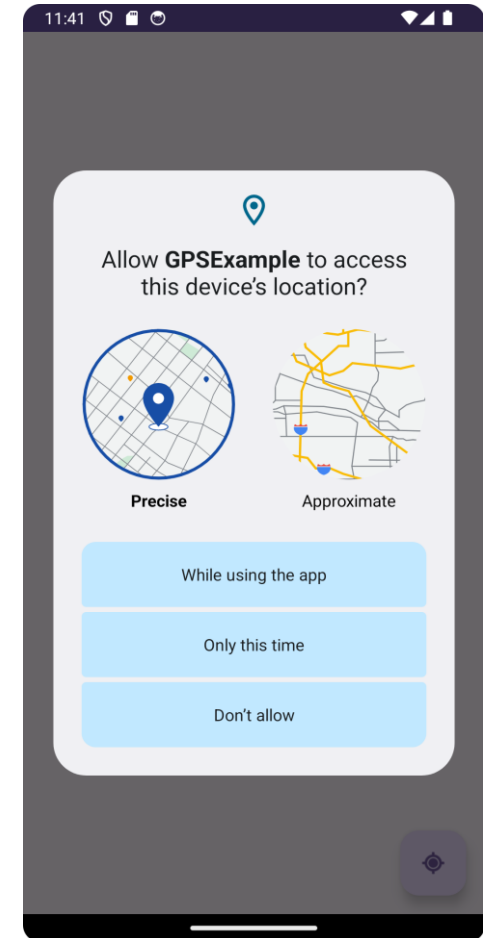
MainActivity.kt

Ejercicio 1

Crea una aplicación que requiera los permisos de localización *coarse* y *fine*.

Al iniciarse la app, comprobará si el usuario ha concedido los permisos necesarios para el funcionamiento de la esta.

1. Si los tiene, mostrará en un campo de texto la palabra "Éxito".
2. Si no los tiene, deberá solicitárselos al usuario:
 - Si el usuario los acepta, se vuelve al punto 1
 - Si los rechaza, se mostrará el mensaje "Fracaso"



Crear un cliente para el LocationProvider

Para **acceder a la localización** del dispositivo hay que **instanciar un cliente de un LocationProvider**. Habitualmente, se utiliza un el proveedor de "fusión", que integra toda la información disponible para ofrecer la ubicación más precisa posible.

```
class MainActivity : AppCompatActivity() {  
    private lateinit var binding: ActivityMainBinding  
    private lateinit var fusedLocationClient: FusedLocationProviderClient  
  
    @SuppressWarnings("MissingPermission")  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = DataBindingUtil setContentView(this, R.layout.activity_main)  
  
        fusedLocationClient =  
        LocationServices.getFusedLocationProviderClient(applicationContext)  
    }  
}
```

MainActivity.kt

Obtener la posición en caché

Una vez el cliente ha sido inicializado, se puede acceder a la posición del móvil.

Hay dos tipos de ubicación disponible, que usaremos dependiendo de las necesidades de la aplicación. Si **no se requiere una localización muy precisa**, podremos utilizar **la última disponible en la caché** del dispositivo y ahorrar así consumo de batería.

```
val posicion = fusedLocationClient.lastLocation.addListener {location ->
    if (location == null) { // GPS apagado
        Toast.makeText(this, "Localización no disponible", Toast.LENGTH_SHORT)
            .show()
    } else {
        Log.d("Location", "%.6f, %.6f".format(location.latitude, location.longitude))
    }
}
```

Obtener la posición actual

Si, por el contrario, **la precisión es relevante en la aplicación a desarrollar**, se puede **forzar a calcular una ubicación precisa y reciente**, en lugar de utilizar la almacenada en caché, que puede estar desfasada.

```
fusedLocationClient.getCurrentLocation(Priority.PRIORITY_HIGH_ACCURACY, object :  
CancellationToken() {  
    override fun onCancelRequested(p0: OnTokenCanceledListener) = CancellationTokenSource().token  
    override fun isCancellationRequested() = false  
})  
.addOnSuccessListener { location: Location? ->  
    if (location == null) { // GPS apagado o petición no resuelta a tiempo  
        Toast.makeText(this, "Localización no disponible", Toast.LENGTH_SHORT).show()  
    } else {  
        Log.d("Location", "%.6f, %.6f".format(location.latitude, location.longitude))  
    }  
}
```


Obtener la posición actual (customizado)

La **configuración por defecto** puede no adecuarse a las necesidades de la app (precisión, velocidad, etc). Podemos **customizar la petición**:

```
val currentLocationRequest = CurrentLocationRequest.Builder()
    .setDurationMillis(1000) // Duración máxima de la petición. Si no, devuelve null
    .setMaxUpdateAgeMillis(2000) // Edad máxima de la posición devuelta
    .setPriority(Priority.PRIORITY_HIGH_ACCURACY) // Relación precisión-consumo
    .setGranularity(Granularity.GRANULARITY_COARSE) // Calidad de la posición obtenida
    .build()
fusedLocationClient.getCurrentLocation(currentLocationRequest, object : CancellationToken() {
    override fun onCancelRequested(p0: OnTokenCanceledListener) = CancellationTokenSource().token
    override fun isCancellationRequested() = false
})
    .addOnSuccessListener { location: Location? ->
        if (location == null) { // GPS apagado o no resuelto a tiempo
            Toast.makeText(this, "Localización no disponible", Toast.LENGTH_SHORT)
                .show()
        } else {
            Log.d("Location", "%.6f, %.6f".format(location.latitude, location.longitude))
        }
    }
}
```

Ejercicio 2

Crea una aplicación que, al pulsar el FAB, muestre por pantalla la posición GPS (latitud y longitud) en la que se encuentra el usuario así como la hora de medición de la localización.

1. Usando la petición por defecto
2. Usando la petición configurada

Su funcionamiento debe comprobarse en el emulador.



Recibir la ubicación periódicamente

Es necesario crear nuevas variables en la actividad para registrar la solicitud de actualizaciones al LocationProvider.

```
import com.google.android.gms.location.LocationCallback
import com.google.android.gms.location.LocationRequest

lateinit var locationRequest: LocationRequest // Requisitos para las actualizaciones
lateinit var locationCallback: LocationCallback // Llamada cuando llega una nueva posición
```

MainActivity.kt

Después, configuramos la solicitud de actualizaciones de ubicación:

```
// Configuración de la solicitud de localización
locationRequest =
    LocationRequest.Builder(1000) // Intervalo al que queremos recibir datos
        .setMinUpdateIntervalMillis(500) // Intervalo más rápido para recibirlos
        .setPriority(Priority.PRIORITY_HIGH_ACCURACY) // Relación precisión-consumo
        .setMinUpdateDistanceMeters(1f) // Distancia entre posiciones para una actualización
        .build()
```

MainActivity.kt

Recibir la ubicación periódicamente (cont.)

Luego, se configura el *callback* a ejecutar cuando una nueva posición de GPS se reciba. Finalmente, se registra la petición.

```
// ... a continuación de lo anterior
// Definición del callback para gestionar la actualización recibida
locationCallback = object : LocationCallback() {
    override fun onLocationResult(locationResult: LocationResult) {
        super.onLocationResult(locationResult)

        // Comprobación extra para ver que nos llega una posición y no viene vacía
        if (locationResult.lastLocation != null){
            val loc = locationResult.lastLocation!!
            Log.d("Location", "%.6f, %.6f".format(loc.latitude, loc.longitude))
        }
    }
}

// Registrar la petición de actualizaciones de localización
fusedLocationClient.requestLocationUpdates(locationRequest, locationCallback, null)
```

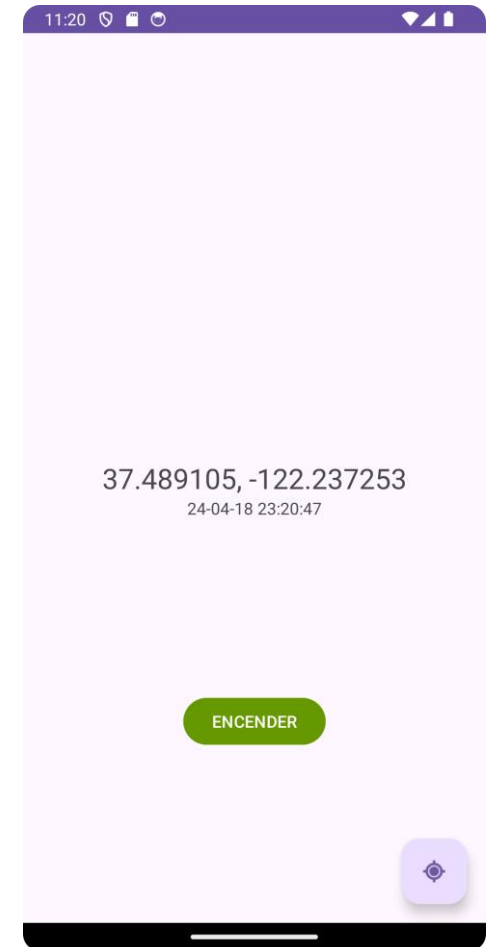
MainActivity.kt

Ejercicio 3

Modifica la aplicación anterior y añade un botón que permita encender y apagar el modo "en directo".

- Cuando la aplicación esté "en directo", la posición mostrada por pantalla se actualiza periódicamente con la llegada de nuevas lecturas de GPS y no únicamente cuando se pulse el FAB.
- Cuando se apague dicho modo, la aplicación funcionará como en el ejercicio anterior.

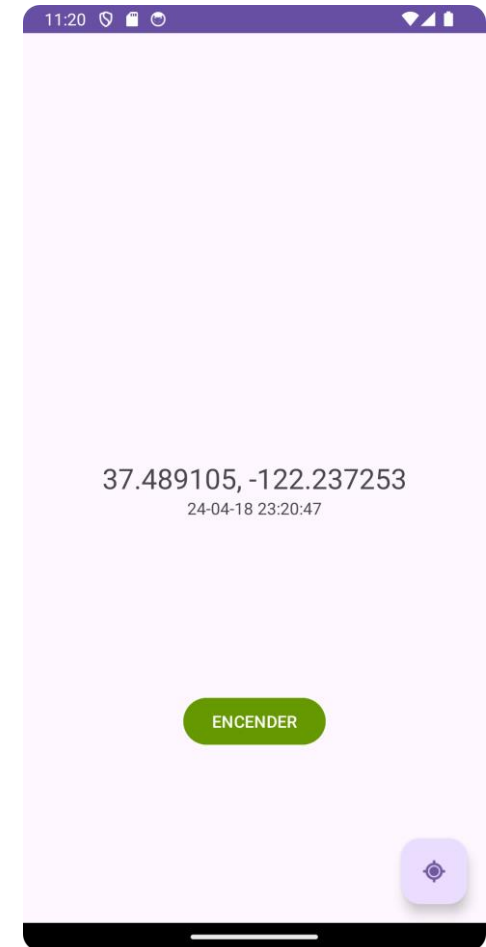
Para parar las actualizaciones de localización sigue el ejemplo de la [documentación oficial](#).



Ejercicio 4 (modo experto)

Partiendo de la aplicación anterior, realiza las modificaciones necesarias para que la ubicación esté guardada en una variable del ViewModel, y la actividad principal observe los cambios sobre la misma.

Para ello, el `FusionLocationProviderClient` y las actualizaciones periódicas o puntuales de la ubicación deberán gestionarse a través del ViewModel, que realizará las operaciones necesarias para obtener la localización y actualizar el objeto *LiveData* correspondiente.



Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

Mapas en Android

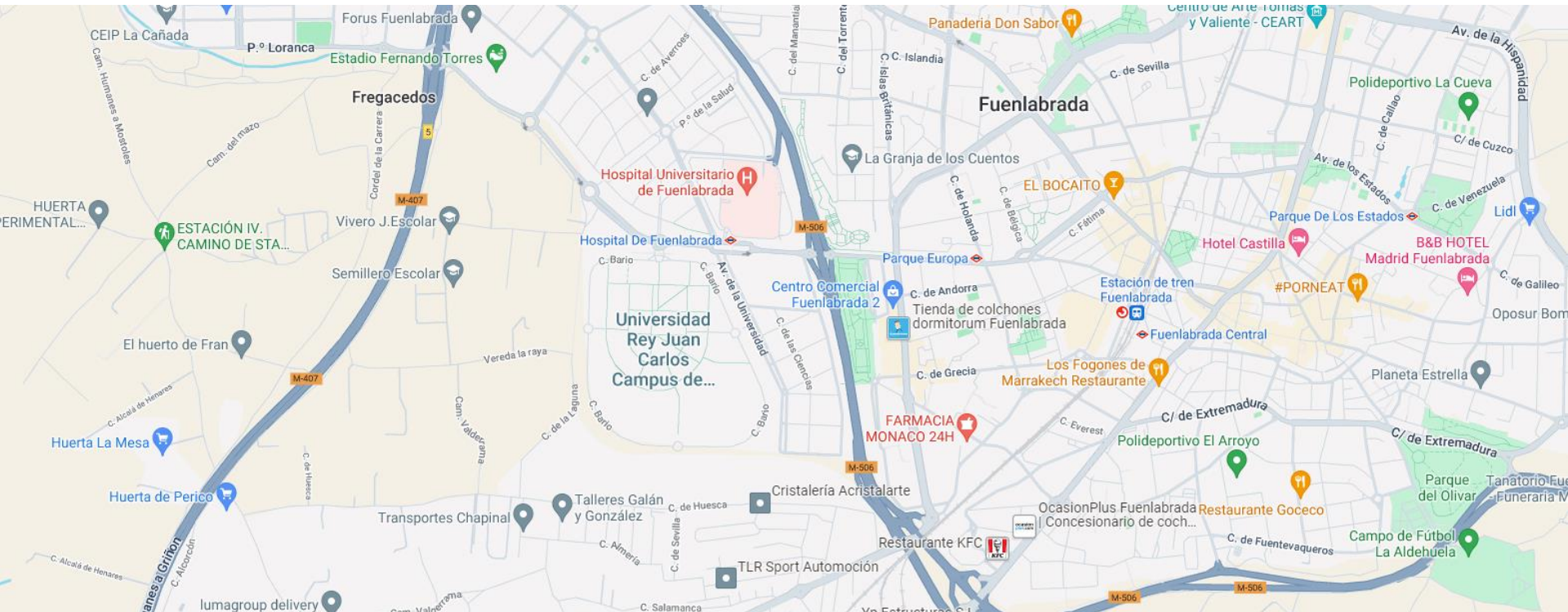


Universidad
Rey Juan Carlos

Mapas en Android

Google Play Services ofrece un conjunto de APIs para Android que permiten utilizar algunas de las tecnologías y servicios más punteros de Google en nuestras aplicaciones, como el [SDK de Google Maps](#), que facilita la inclusión de mapas.

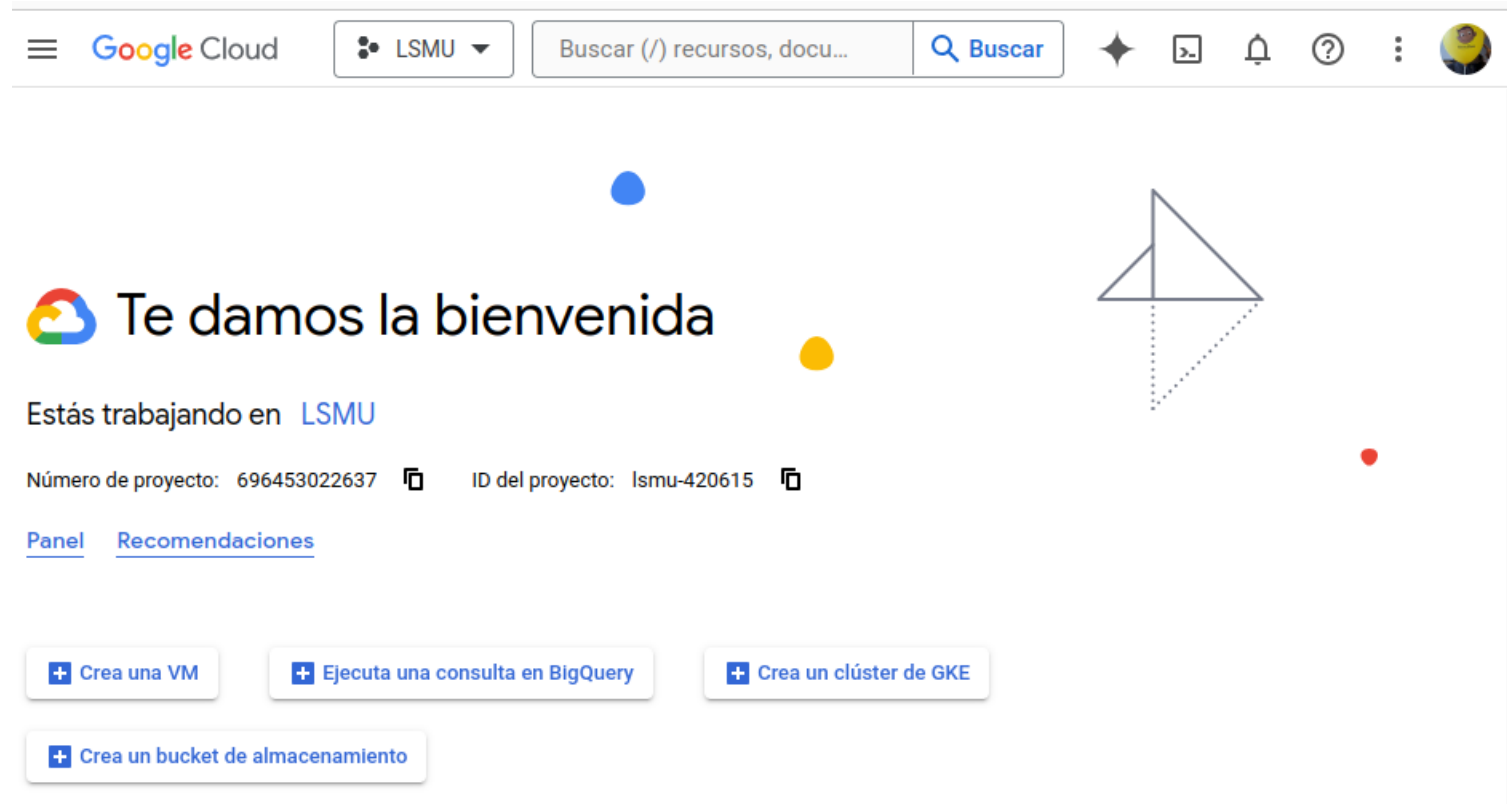
Aunque es el más popular, existen alternativas: Mapbox, Open Street Maps...



Google Cloud Console

Google Console

Para utilizar Google Play Services (y, por tanto, Maps) debemos tener una cuenta en Google y acceder a la [consola de Google Cloud](#).



Configurar facturación

La API de Google Maps es gratuita hasta agotar los créditos gratuitos mensuales. Superado ese límite, tiene un coste.

Para poder empezar a usarla, debemos configurar los datos de facturación de nuestra cuenta introduciendo un método de pago.



The image shows a screenshot of the Google Cloud console interface. On the left, the navigation menu includes 'Google Cloud', 'LSMU', and several options: 'Descripción general de Cl...', 'Descripción general de Cloud', 'PRODUCTOS FIJADOS', 'API APIs y servicios', and 'Facturación'. The 'Facturación' option is highlighted with a red rectangular box. The main content area is titled 'Facturación' and has two tabs: 'MIS CUENTAS DE FACTURACIÓN' (selected) and 'MIS PROYECTOS'. Under the selected tab, there is a section titled 'Facturación Cuentas' with the text: 'Agrega una cuenta de facturación para acceder al con... servicios y a un incremento en los límites de uso. [Más informac...](#)'. Below this text is a blue button labeled 'AGREGAR CUENTA DE FACTURACIÓN'. In the bottom right corner, there is a meme featuring the Pokémon Squirtle with the text 'VAMO A CALMARN' overlaid.

Crear un proyecto

Para tener acceso al Google Maps SDK, es necesario crear un proyecto.

Buscar

Proyecto nuevo



Tienes 21 projects restantes en tu cuota. Solicita un incremento o borra algunos proyectos. [Más información](#)

[MANAGE QUOTAS](#)

Nombre del proyecto *



ID del proyecto: trabajofinal-421207. No se puede cambiar más adelante. [EDITAR](#)

Ubicación *

 Sin organización [EXPLORAR](#)

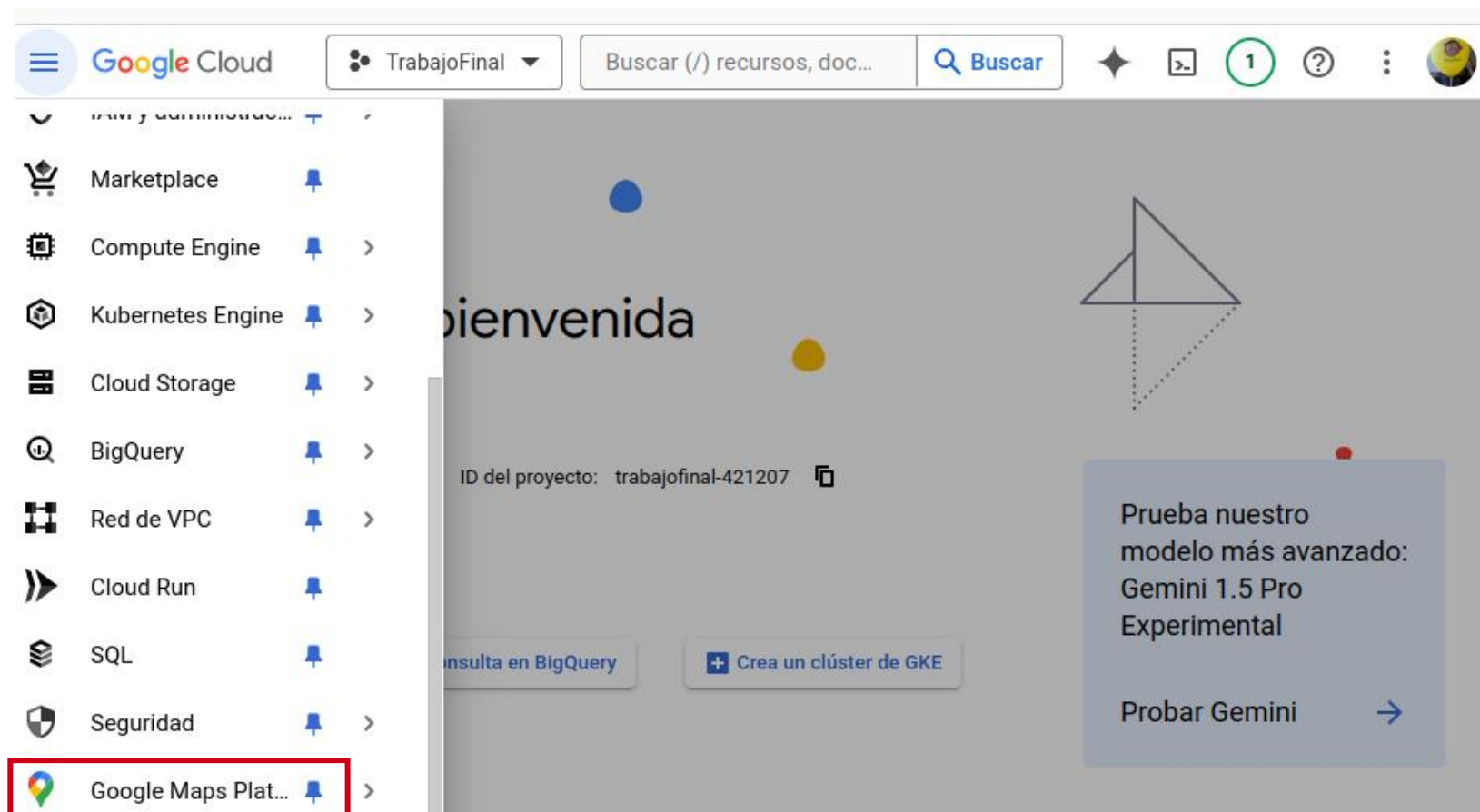
Organización o carpeta superior

CREAR

CANCELAR

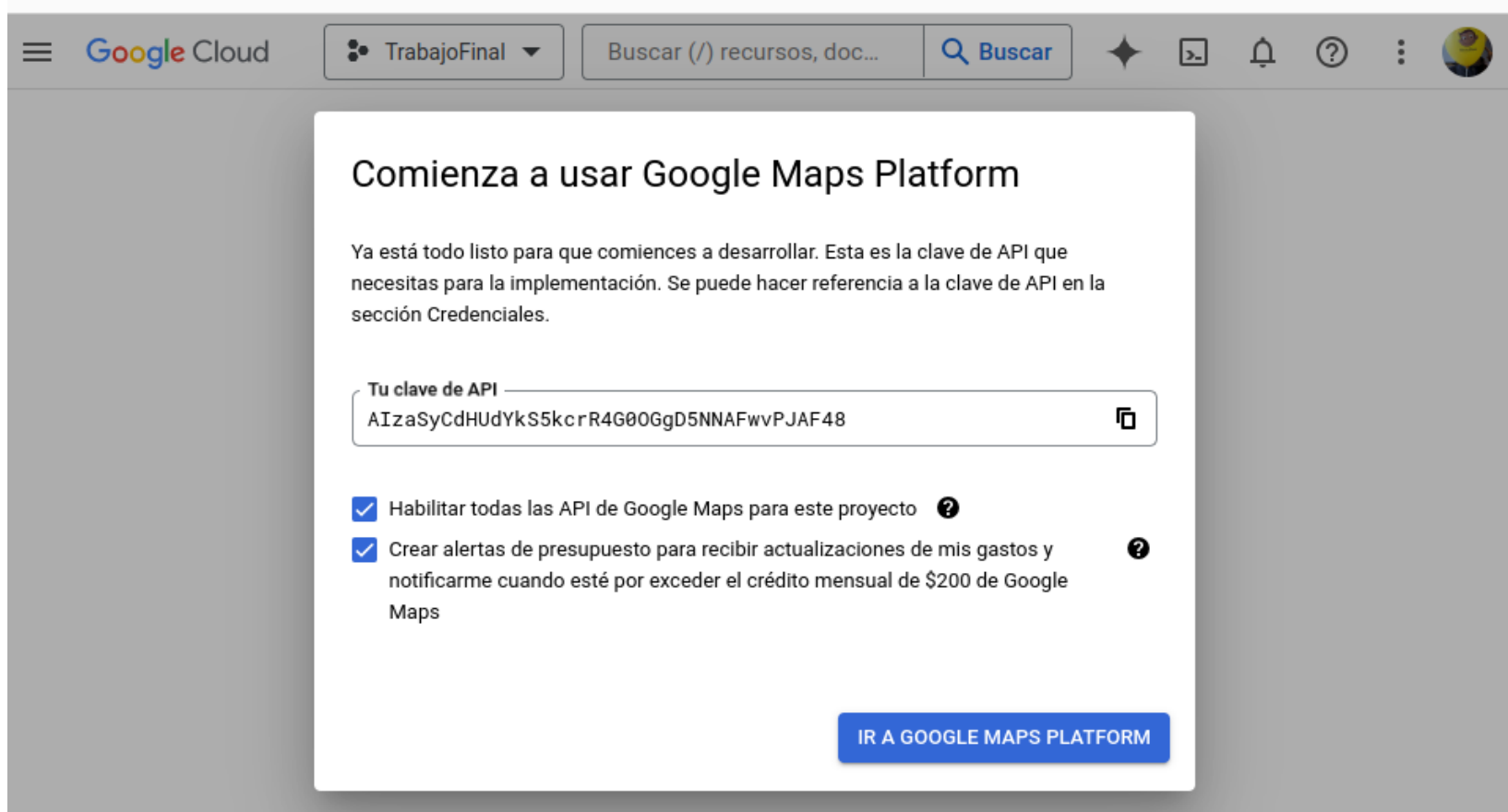
Añadir Google Maps

Una vez creado el proyecto, hay que añadir el SDK de Google Maps para poder habilitar la API correspondiente.



Google Maps API Key

Al añadirlo, se genera una clave para autenticar nuestra aplicación cuando haga uso de la API. También se puede ver desde: *Google Maps Platform > Claves y Credenciales*



Habilitar el SDK

En *APIs y servicios* se pueden habilitar aquellas APIs necesarias para nuestro proyecto. En nuestro caso, habilitaremos el *SDK para Android*

Google Cloud

TrabajoFinal

Buscar (/) recursos, doc...

Buscar

Google Maps Platform

APIs y servicios

APRENDIZAJE

Descripción general

API APIs y servicios

Métricas

Cuotas

Claves y credenciales

Asistencia

Biblioteca de soluciones

Administración de mapas

Diseños de mapa

Notas de versión

Maps JavaScript API

Maps for your website

MAPS

Maps SDK for Android

Maps for your native Android app.

MAPS

Guides

Maps SDK for iOS

ENABLE

Habilitamos únicamente esta API (click en *Enable* y aparece *Disable*). Las demás deben deshabilitarse (click en *Disable*) si vienen habilitadas por defecto

Establecer cuotas para evitar gastos

Limitar las consultas a la API evita incurrir en gastos de facturación.

Google Cloud

TrabajoFinal

Buscar (/) recursos, doc...

Buscar

Google Maps Platform

Cuotas

Maps SDK for Android

INHABILITAR

APRENDIZAJE

Descripción general

APIs y servicios

Métricas

Cuotas

Claves y credenciales

Asistencia

Biblioteca de soluciones

Administración de mapas

Administrar recursos

Notas de versión

Para solicitar más límites de cuota o consultar las cuotas de tus otros servicios, visita la [página Cuotas](#), que se encuentra en IAM y administración.

Las cuotas diarias se restablecen a medianoche, hora del Pacífico (PT).

Street View Requests

Map Loads

Nombre de la cuota	Límite
Paid Map Loads por día	0
Paid Map Loads por minuto	0
Paid Map Loads por minuto por usuario ?	0

Administrar las credenciales

Aunque la clave API nunca debe hacerse pública, es conveniente restringir su uso para evitar problemas de seguridad.

Google Cloud

TrabajoFinal

Buscar (/) recursos, doc...

Buscar

Google Maps Platform

Claves y credenciales

Maps SDK for Android

+ CREAR CREDENCIALES

■ INHABILITAR

Descripción general

APIs y servicios

Métricas

Cuotas

Claves y credenciales

Asistencia

Biblioteca de soluciones

Administración de mapas

Diseños de mapa

Credenciales compatibles con esta API

Para ver todas las credenciales, visita [Credenciales en la sección API y servicios](#)

⚠ Recuerda configurar la pantalla de consentimiento de OAuth con información sobre tu app.

[CONFIGURAR PANTALLA DE CONSENTIMIENTO](#)

Claves de API

Nombre	Fecha de creación	Restricciones	Acciones
⚠ Maps API Key	23 abr 2024	Ninguno	MOSTRAR CLAVE

Restringir el uso de la clave API

Google Cloud

TrabajoFinal

Buscar (/) recursos, documentos, productos y más

Buscar

API APIs y servicios

APIs y servicios habilitados

Biblioteca

Credenciales

Pantalla de consentimiento ...

Acuerdos de uso de páginas

Editar clave de API

VOLVER A GENERAR CLAVE

BORRAR

Nombre *

Maps API Key

Restricciones de clave

Esta clave no tiene restricciones. A fin de evitar el uso no autorizado, te recomendamos restringir dónde y para qué API se puede usar.[Más información](#)

Establece una restricción de aplicaciones

Las restricciones de aplicaciones limitan el uso de las claves de API a sitios web, direcciones IP y aplicaciones para Android o iOS específicas. Puedes configurar una restricción de aplicaciones por clave.

☐ Ninguno

☐ Sitios web

☐ Direcciones IP

☒ Apps para Android

☐ Apps para iOS

Restricciones de Android

Restringe el uso de la clave de API a las apps para Android especificadas. Agrega el nombre del paquete y la huella digital del certificado SHA-1 para cada app.

+ ADD

Filtro

Ingresar el nombre o el valor de la propiedad

☐

Estado

☐

Nombre del paquete

☐

Huella digital

☐

Editar

No hay filas para mostrar

Información adicional

API Key

AIzaSyCdHudYkS5kcrR4G8OGgD5NNAFwvPJAF48

Para usar esta clave en tu aplicación, debes transferirla con el parámetro `key=API_KEY`.

Fecha de creación

23 de abril de 2024, 09:39:34 GMT+2

¿Cómo puedo restringir mi clave de API a aplicaciones para Android específicas?

A fin de restringir una clave de API a aplicaciones para Android específicas, proporciona la huella digital de un certificado de depuración o la de un certificado de lanzamiento.

Huella digital del certificado de depuración

Para Linux o macOS:

\$ keytool -list -v -keystore ~/.android/debug.keystore -alias and

Para Windows:

\$ keytool -list -v -keystore "%USERPROFILE%\android\debug.keystore

Huella digital del certificado de lanzamiento

\$ keytool -list -v -keystore your_keystore_name -alias your_alias

Reemplaza `your_keystore_name` por la ruta de acceso completamente calificada y el nombre del almacén de claves, incluida la extensión `.keystore`. Reemplaza `your_alias_name` por el alias con el que creaste el certificado.

12 Jorge Beltrán

Lab. de Sistemas Móviles y Ubicuos · Universidad Rey Juan Carlos · Curso 2024/2025

Restringir la clave API a una sola app

Para mayor seguridad, se debe restringir la clave para que solo pueda usarse desde una sola aplicación (nombre de paquete + huella digital).

¿Cómo puedo restringir mi clave de API a aplicaciones para Android específicas?

A fin de restringir una clave de API a aplicaciones para Android específicas, proporciona la huella digital de un certificado de depuración o la de un certificado de lanzamiento.

Huella digital del certificado de depuración

Para Linux o macOS:

```
$ keytool -list -v -keystore ~/.android/debug.keystore -alias and
```

Para Windows:

```
$ keytool -list -v -keystore "%USERPROFILE%\android\debug.keystore"
```

Huella digital del certificado de lanzamiento

```
$ keytool -list -v -keystore your_keystore_name -alias your_alias
```

Reemplaza your_keystore_name por la ruta de acceso completamente calificada y el nombre del almacén de claves, incluida la extensión .keystore. Reemplaza your_alias_name por el alias con el que creaste el certificado.

☒ Apps para Android

☐ Apps para iOS

Restricciones de Android

Restringe el uso de la clave de API a las apps para Android especificadas. Agrega el nombre del paquete y la huella digital del certificado SHA-1 para cada app.

Agregar una aplicación para Android

Nombre del paquete *
com.lsmu.trabajofinal

Huella digital del certificado SHA-1 *
91:51:C5:FA:96:9A:56:B8:CB:9F:37:F7:AA:89:68:40:2B:20:AD:63

CANCELAR

LISTO

Maps SDK en Android

API Key segura con *secrets plugin*

Añadir dependencias del plugin [secrets](#) y (opcional) especificar los archivos donde se definen las claves. Por defecto en *local.properties*.

```
buildscript {  
    dependencies {  
        classpath("com.google.android.libraries.mapsplatform.secrets-gradle-plugin:2.0.1")  
    }  
}
```

build.gradle.kts (Project)

```
plugins {  
    id("com.google.android.libraries.mapsplatform.secrets-gradle-plugin")  
}  
  
secrets { // OPCIONAL PARA CONFIGURACIÓN AVANZADA  
    propertiesFileName = "secrets.properties" // Por defecto en local.properties  
    defaultPropertiesFileName = "local.defaults.properties" // Secretos por defecto, subibles al repo  
}
```

build.gradle.kts (:app)

Agregar la clave de API al archivo de secretos del proyecto:

```
GOOGLE_MAPS_API_KEY=TU_CLAVE_SIN_COMILLAS
```

local.properties

Añadir la API Key al Manifest

Una vez configurado el plugin *secrets*, es necesario añadirla en el manifiesto, indicando que es la `API_KEY` del SDK y referenciando el identificador que le hemos dado a la clave en el archivo de secretos.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:tools="http://schemas.android.com/tools">

  <application
    >
    <meta-data
      android:name="com.google.android.geo.API_KEY"
      android:value="{GOOGLE_MAPS_API_KEY}" />

    <activity
      ... />
  </application>

</manifest>
```

AndroidManifest.xml

Dependencias y elemento en la UI

Para mostrar un mapa de Google en una app, se debe añadir sus dependencias correspondientes.

```
dependencies {  
    implementation("com.google.android.gms:play-services-maps:18.2.0")  
    implementation("com.google.maps.android:android-maps-utils:2.2.0")  
    ...  
}
```

build.gradle.kts (:app)

Luego, se debe añadir el componente a nuestra interfaz. Para insertarlo en una actividad, se puede utilizar el elemento *MapView*.

```
<com.google.android.gms.maps.MapView  
    android:id="@+id/mapView"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:visibility="invisible"/>
```

activity_main.xml

Configurar MapView en la actividad

Hay que "enlazar" el ciclo de vida del mapa al de la actividad (*onCreate*, *onResume*, etc). Luego, inicializar una variable de tipo *GoogleMap* cuando el *MapView* haya cargado.

MainActivity.kt

```
private lateinit var googleMap: GoogleMap

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = DataBindingUtil setContentView(this,
R.layout.activity_main)
    binding.mapView.onCreate(savedInstanceState)

    // Inicializar googleMap cuando el MapView cargue
    binding.mapView.getMapAsync { map ->
        googleMap = map // Para usarlo en otros sitios
        googleMap.setOnMapLoadedCallback {
            // Código para cuando el mapa está cargado
        }
    }
}
```

Si no están todas las funciones
onXXXX implementadas, falla!

MainActivity.kt

```
override fun onResume() {
    super.onResume()
    binding.mapView.onResume()
}

override fun onPause() {
    super.onPause()
    binding.mapView.onPause()
}

override fun onDestroy() {
    super.onDestroy()
    binding.mapView.onDestroy()
}

override fun onLowMemory() {
    super.onLowMemory()
    binding.mapView.onLowMemory()
}
```

Gestionar la cámara del MapView

La fracción de mapa que se muestra en el componente MapView es la "cámara".

Esta cámara se puede configurar para centrarla en una posición determinada, con el nivel de zoom deseado.

También se pueden obtener detalles de la cámara a través del atributo *cameraPosition* disponible en el objeto de tipo *GoogleMap*.

```
val latLng = LatLng(13.411593, 103.867416)
val cameraUpdate = CameraUpdateFactory.newLatLng(latLng)
googleMap.moveCamera(cameraUpdate)
```

MainActivity.kt



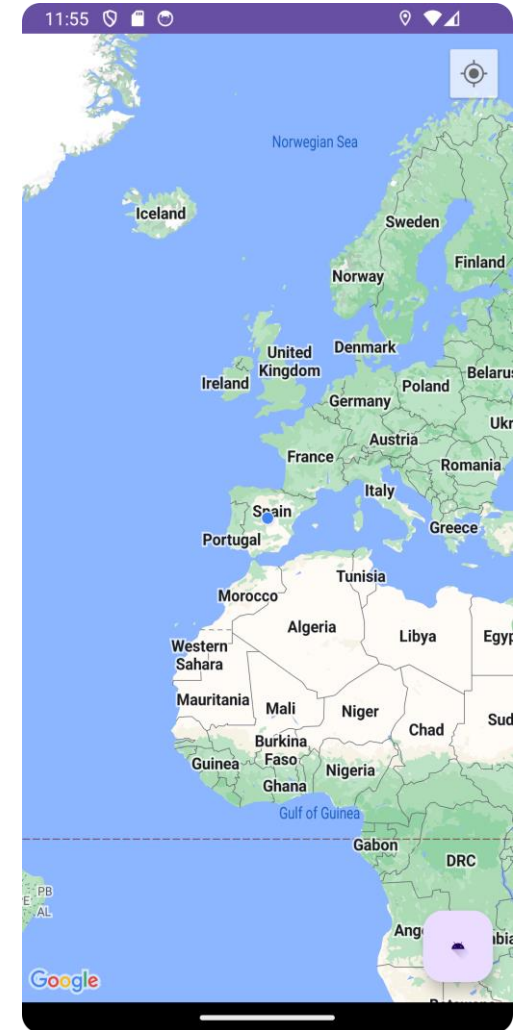
Se pueden especificar también otros parámetros con el resto de funciones de nombre *newXXXX*

Ejercicio 1

Crea una aplicación que muestre un MapView y un botón flotante que permitirá mover la cámara del entre localizaciones predefinidas.

La aplicación debe contar con una lista de posiciones GPS prefijadas en las capitales de Moscú, París, Londres y Bogotá.

Al iniciar la aplicación, el mapa debe posicionarse sobre la primera ciudad de la lista. Cuando se pulse el FAB, el mapa debe apuntar a la siguiente localización.



Añadir marcadores al mapa

Usando el objeto de tipo *GoogleMap*, se pueden añadir marcadores al mapa.

Un marcador sencillo puede crearse indicando una posición GPS. Sin embargo, estos se pueden personalizar con título, icono, rotación, y [más atributos](#).

```
googleMap.addMarker(  
    MarkerOptions(  
        .title("Mi lugar favorito")  
        .position(LatLng(location.latitude, location.longitude))  
    )  
)
```

MainActivity.kt

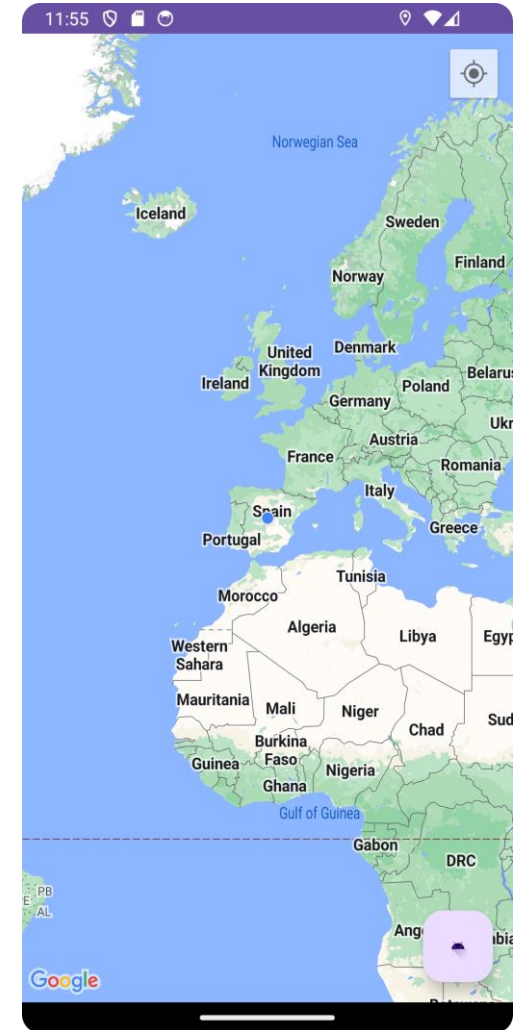


Ejercicio 2

Partiendo de la aplicación anterior, añade un botón en la parte inferior de la aplicación para poder añadir a la lista de ciudades una nueva localización.

Al pulsar este botón, se obtendrá la posición a la que apunta la cámara del MapView y se guardará en la lista. Además, se añadirá un Marcador en dicha posición del mapa.

El botón flotante deberá mantener su funcionalidad para poder movernos entre los lugares guardados en la lista.

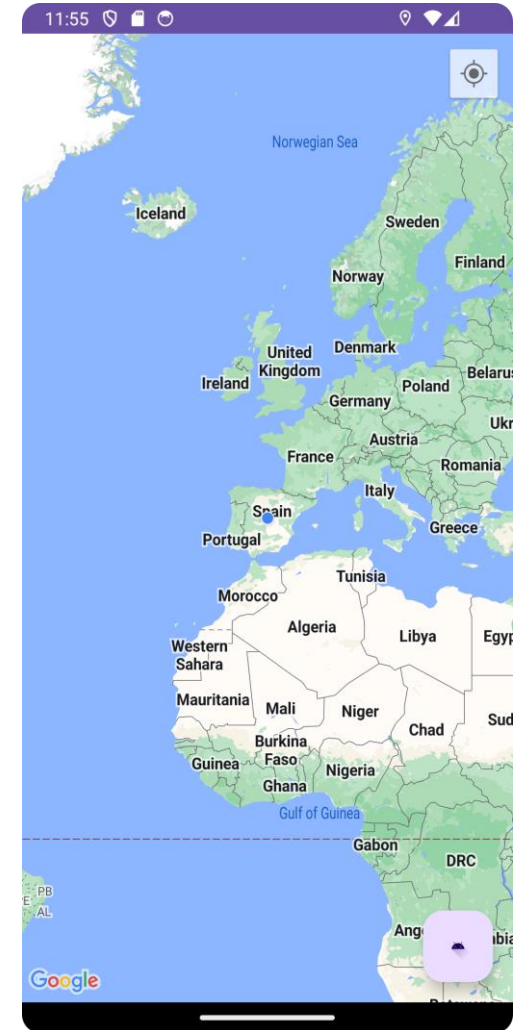


Ejercicio 3

Partiendo de la aplicación anterior, añada un [listener](#) al mapa para poder añadir nuevas localizaciones a la lista ciudades pulsando sobre lugares del propio mapa.

Cuando el usuario haga click sobre el mapa, se deberá insertar un marcador en dicha posición, añadiendo, además, la posición marcada a la lista de ciudades por las que podemos movernos.

El resto de las funcionalidades de la app deben preservarse.



Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>